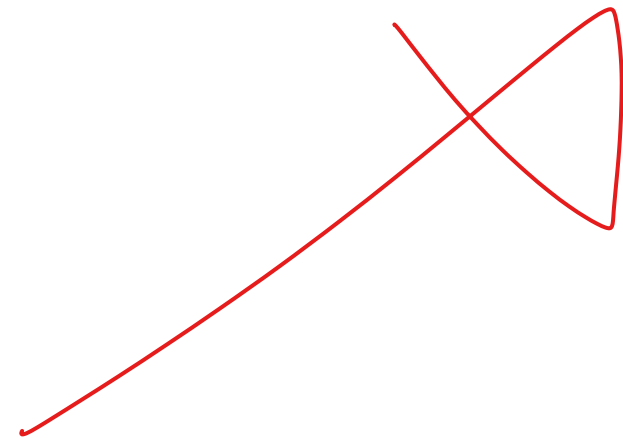


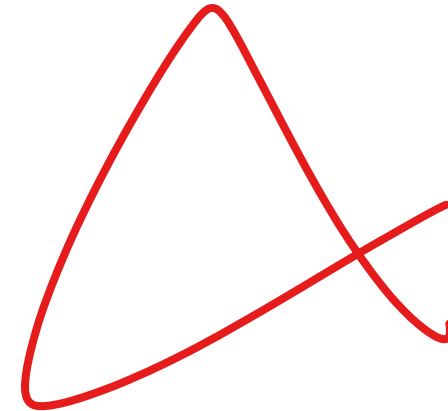
Tree



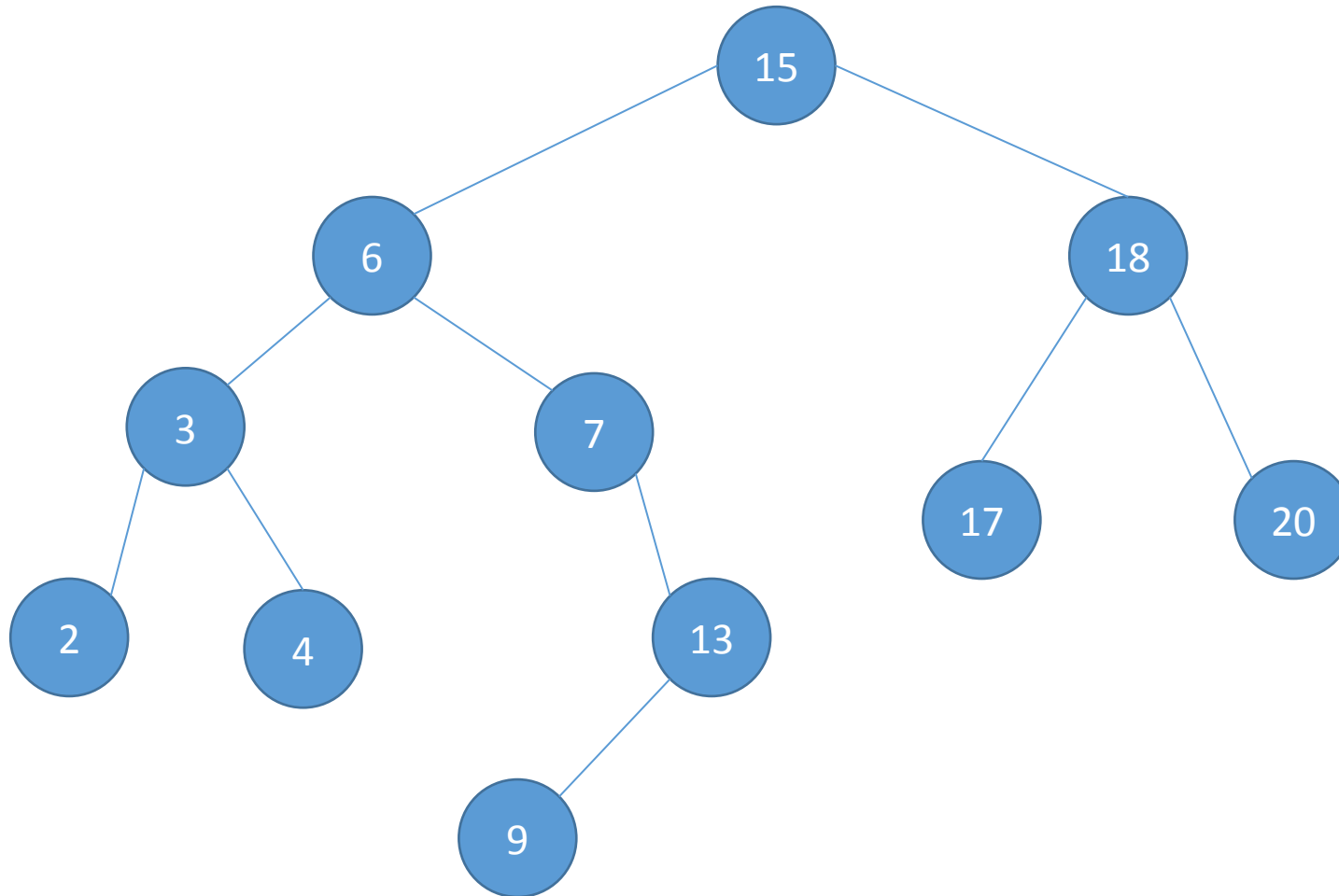
TREE PART III

Binary Search Tree

- Insert Function
- Delete Function
- Discussing Complexity

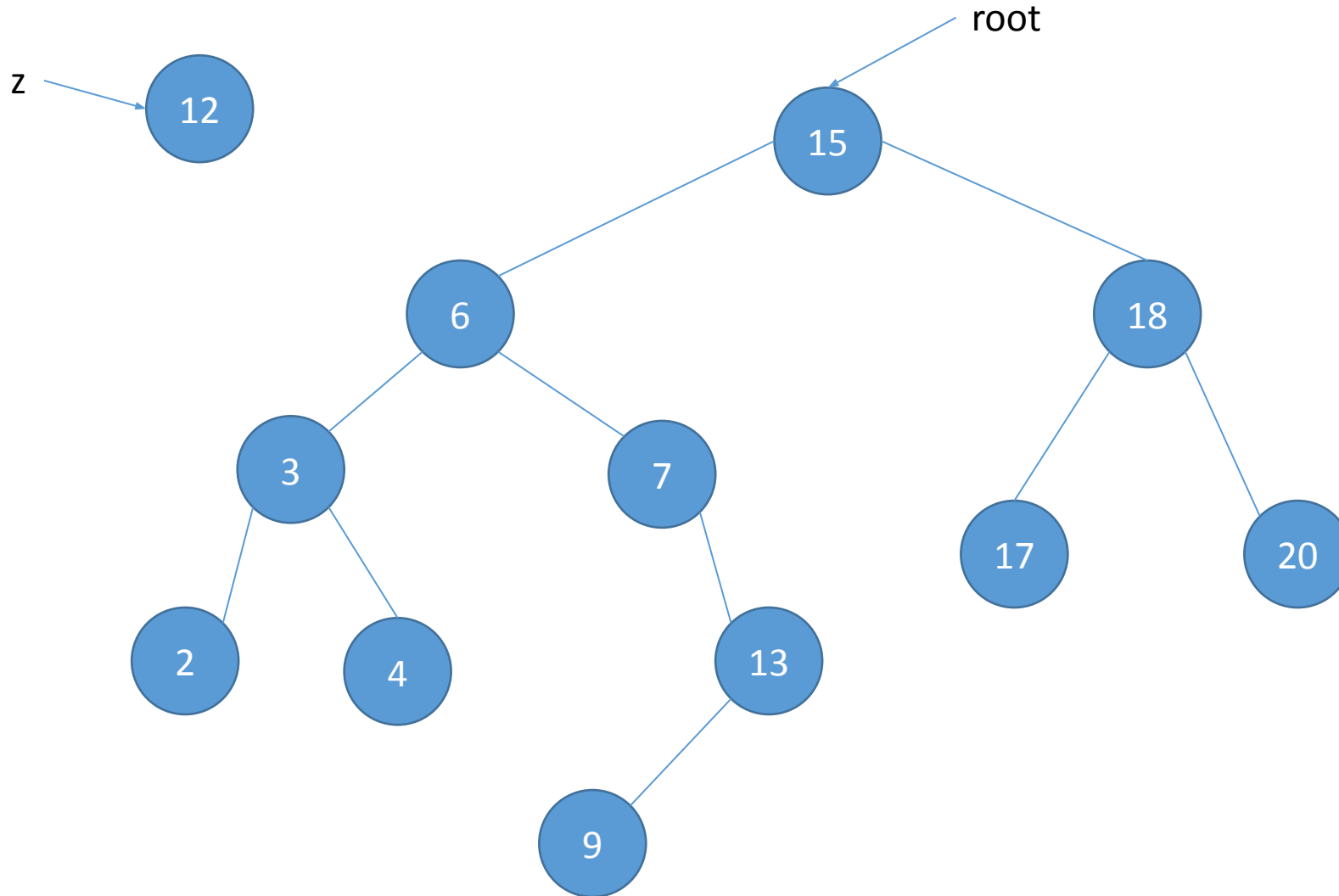


Insert Function



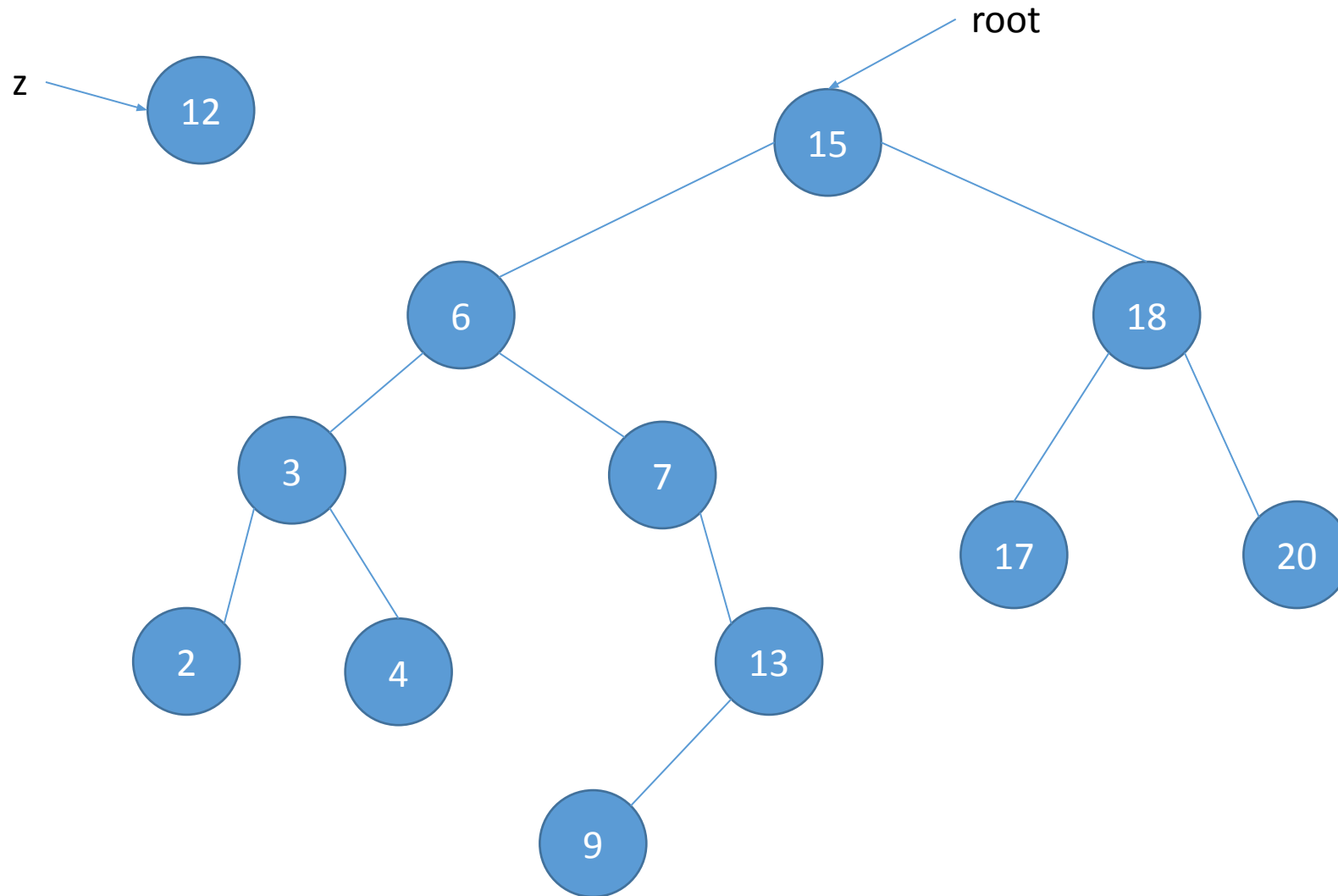
- Let us **insert 12**
 - We need to find the **position** for the new Tree Node with the value 12
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (**i.e it is connected**)

Insert Function



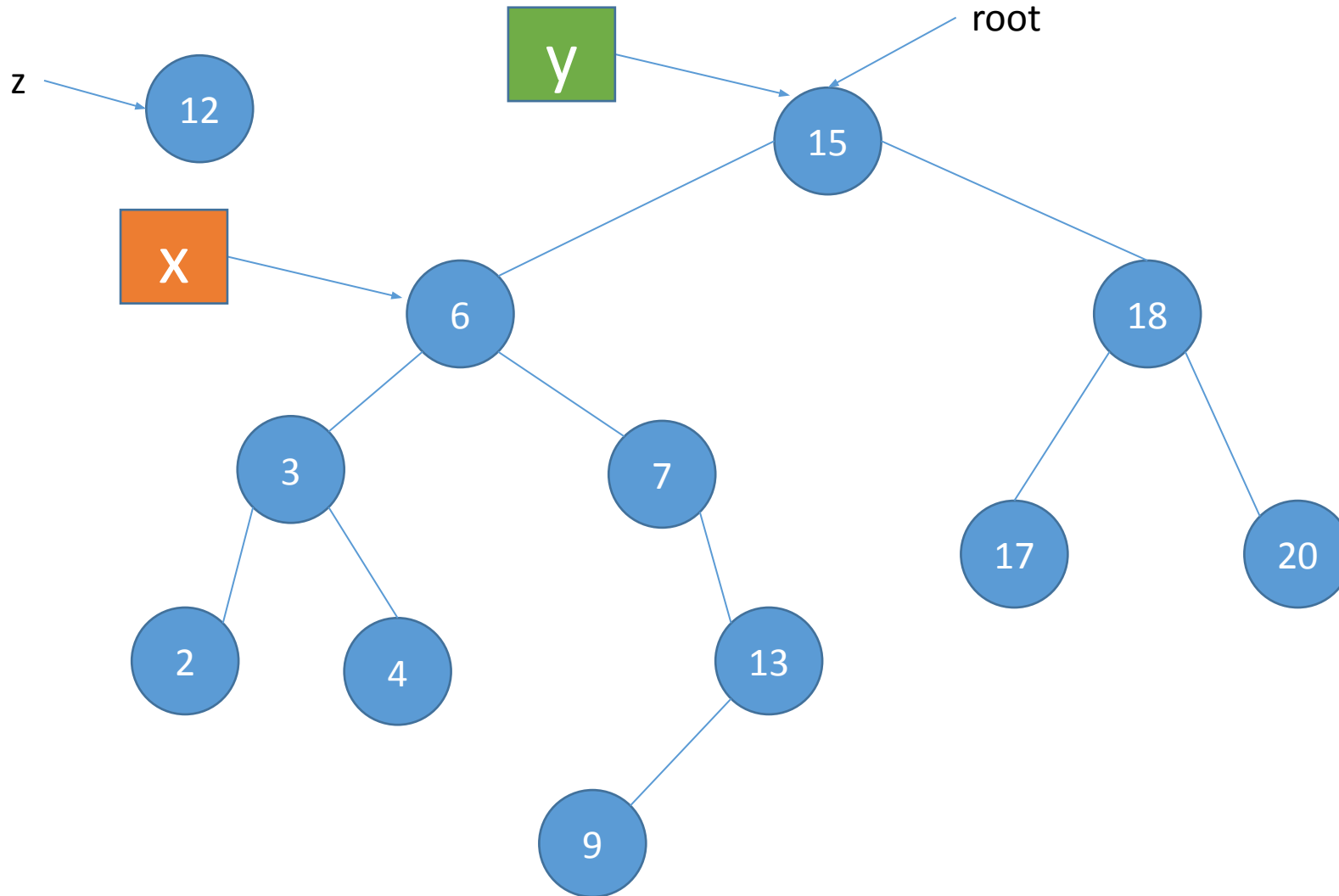
- Let us insert 12
 - We need to find the **position** for the new Tree Node with the value 12
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (**i.e it is connected**)

Insert Function



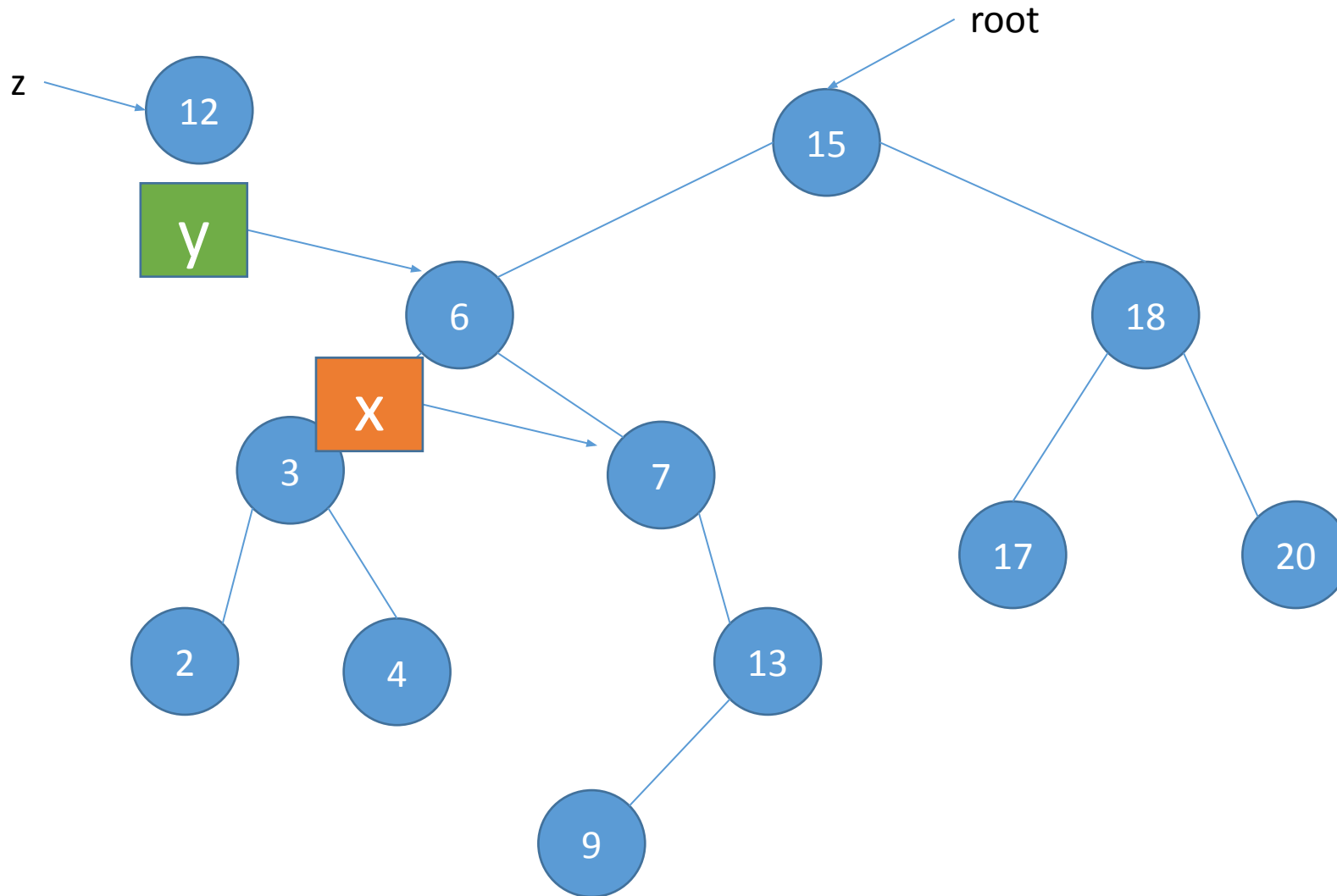
- Let us insert 12
 - We need to find the **position** for the new Tree Node with the value 12 **X**
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (i.e it is **Y** connected)

Insert Function



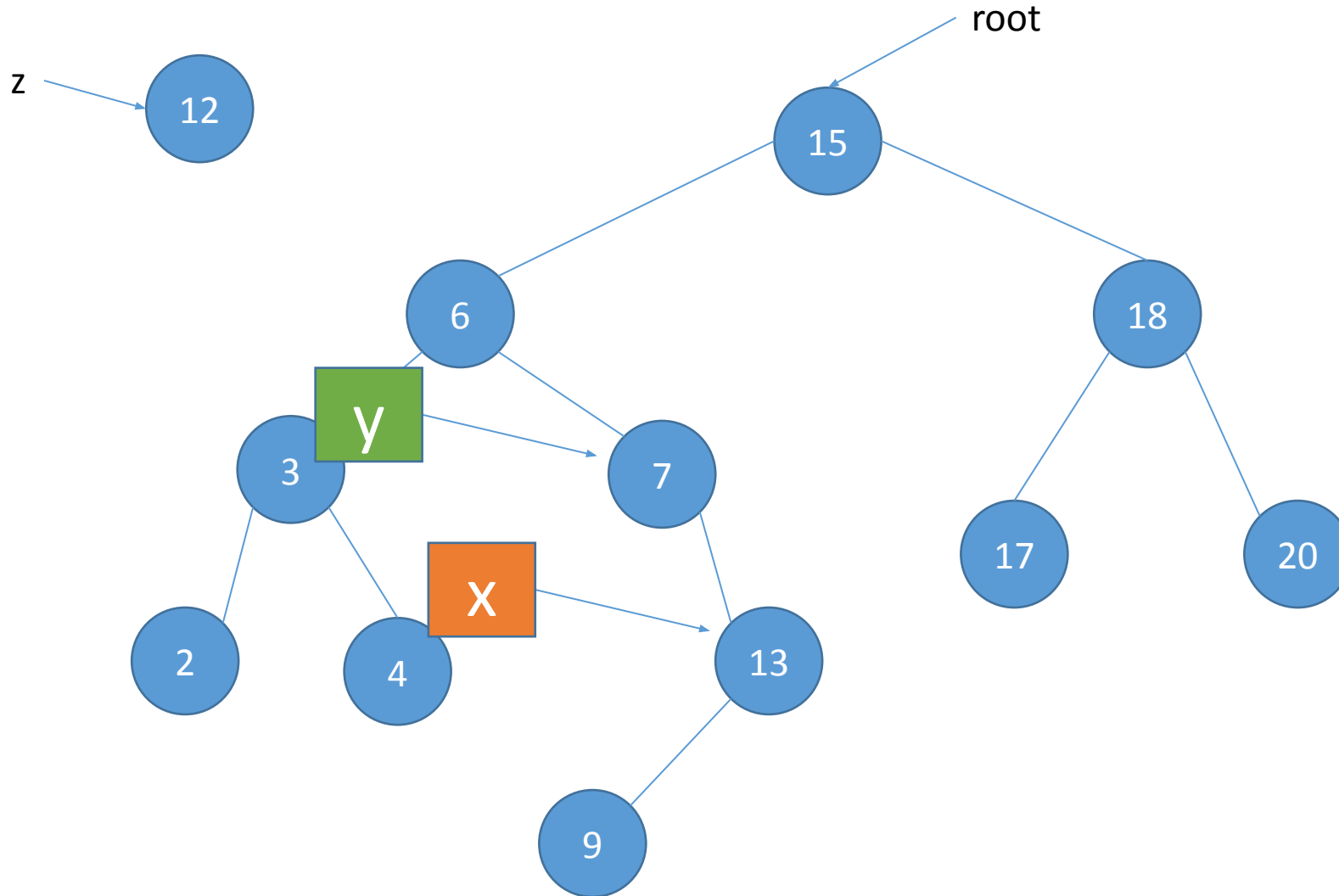
- Let us insert 12
 - We need to find the **position** for the new Tree Node with the value 12 **X**
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (i.e it is **Y** connected)

Insert Function



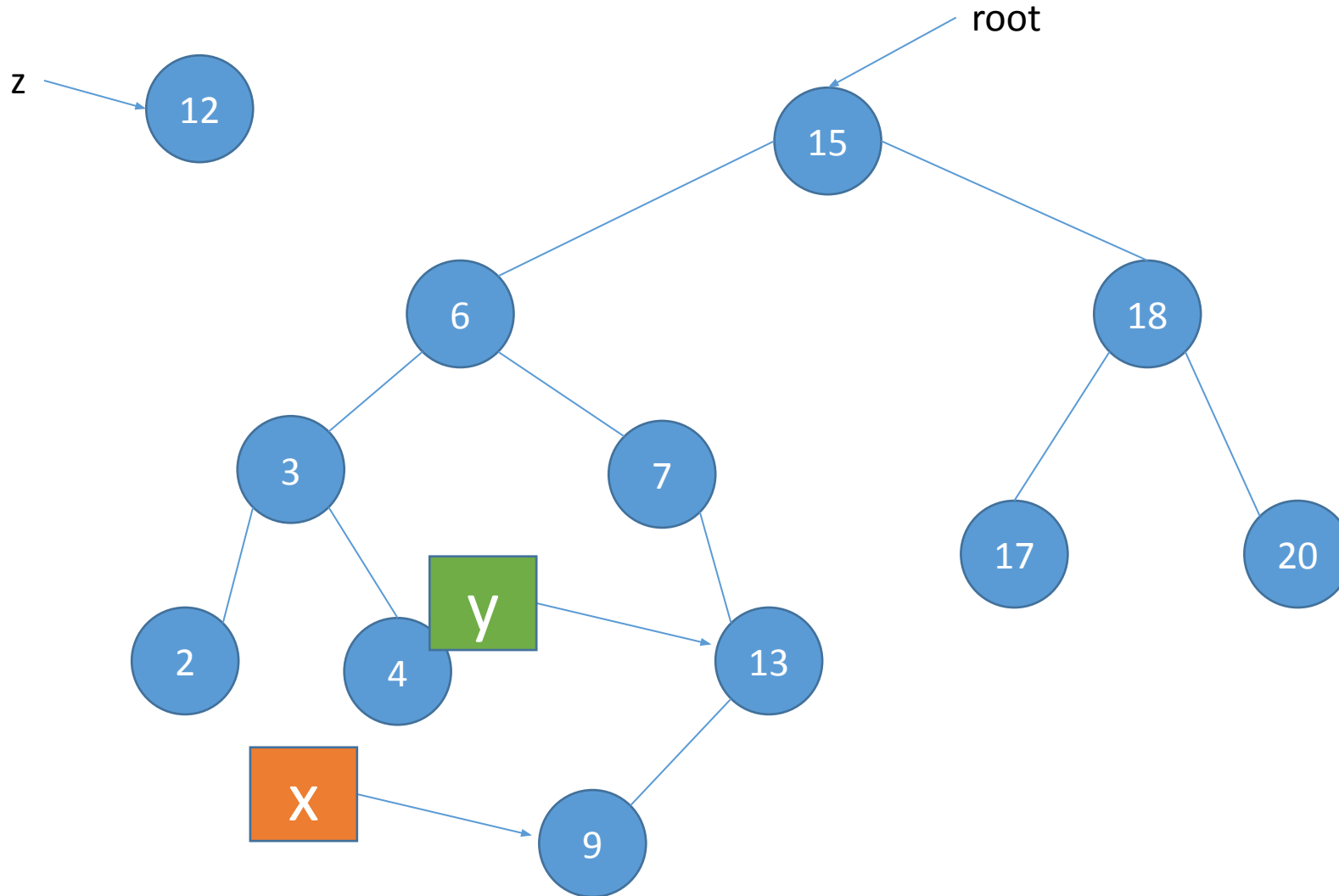
- Let us insert 12
 - We need to find the **position** for the new Tree Node with the value 12 **X**
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (i.e it is **Y** connected)

Insert Function



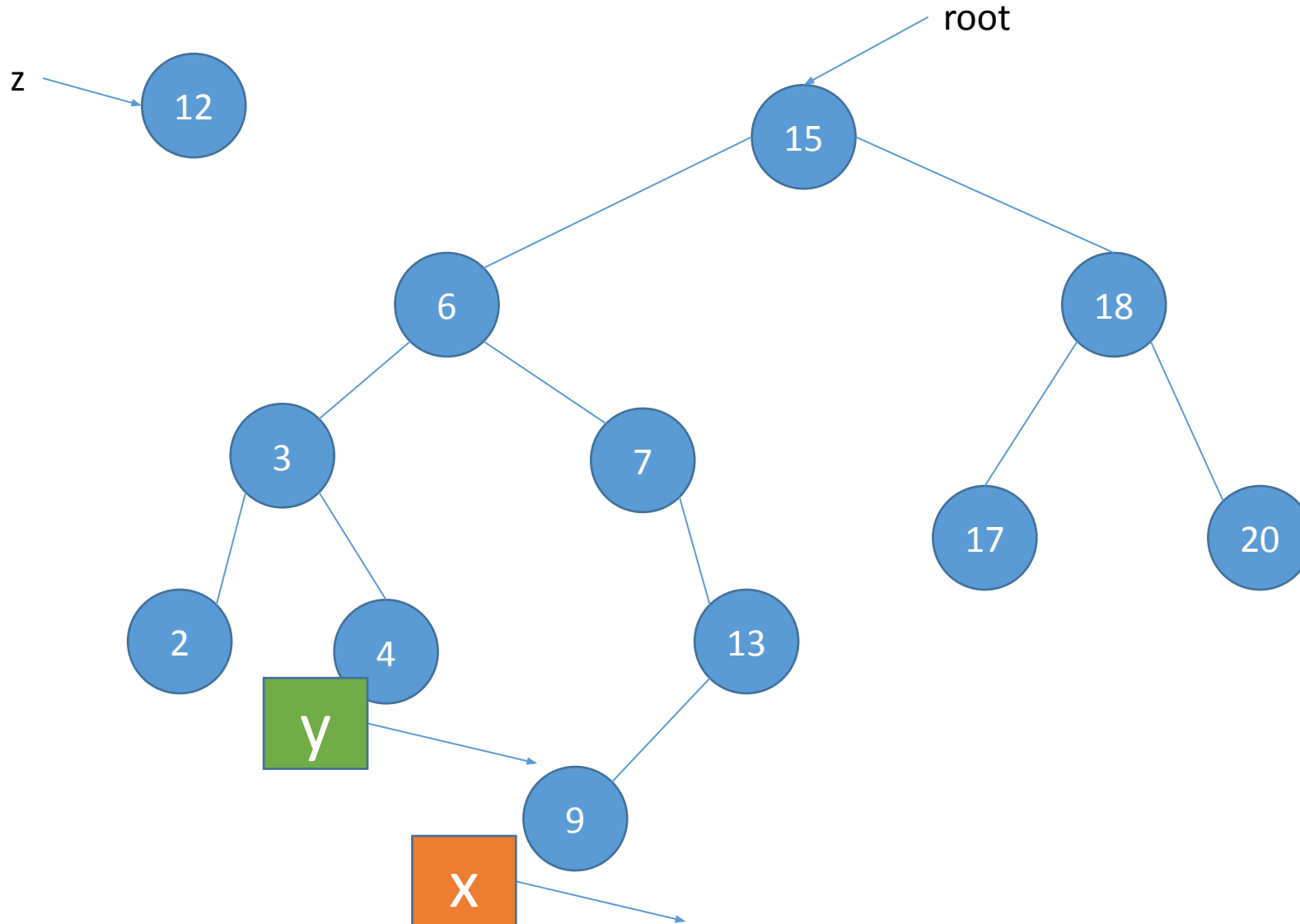
- Let us insert 12
 - We need to find the **position** for the new Tree Node with the value 12 **X**
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (i.e it is **Y** connected)

Insert Function



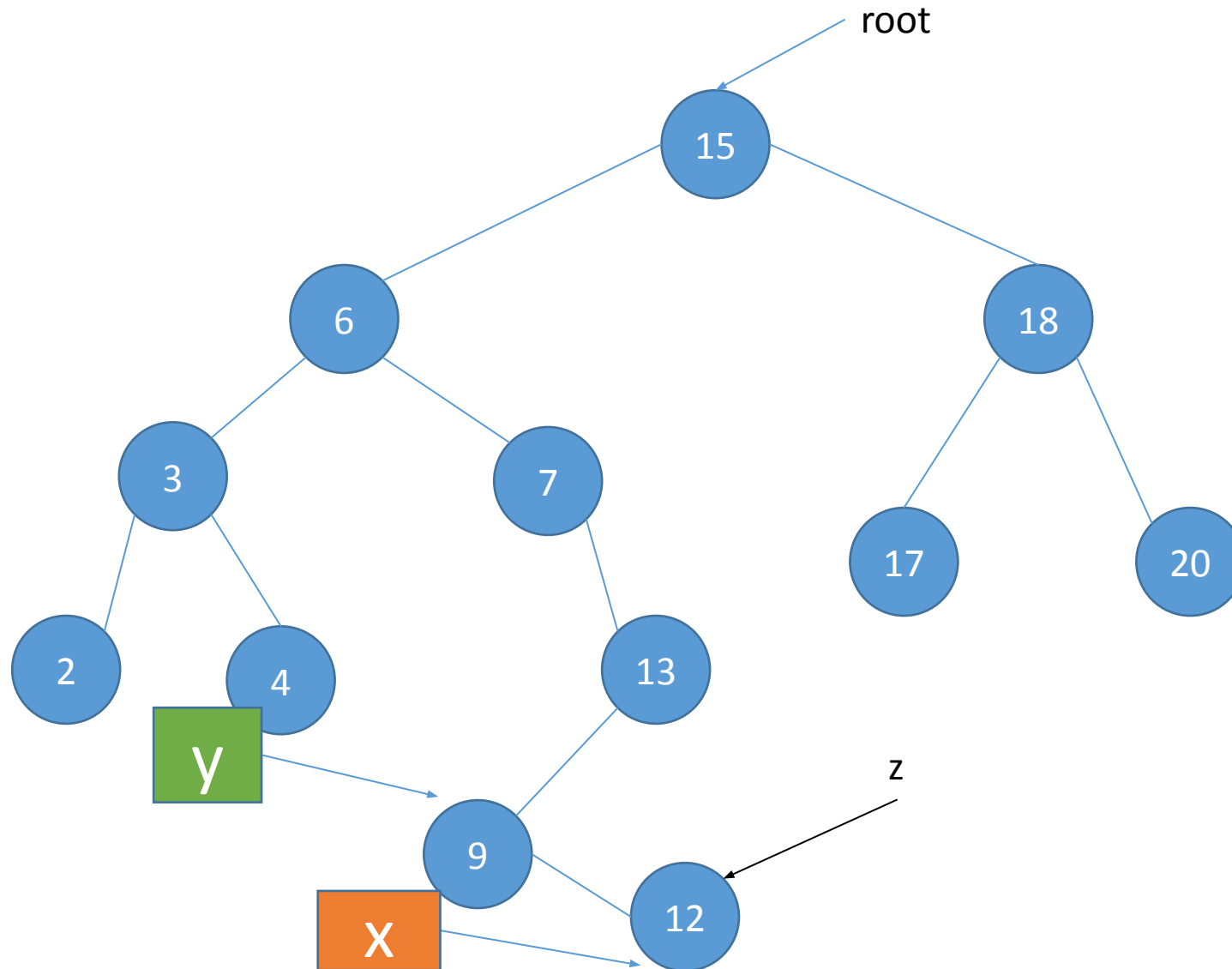
- Let us insert 12
 - We need to find the **position** for the new Tree Node with the value 12 **X**
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (i.e it is **Y** connected)

Insert Function



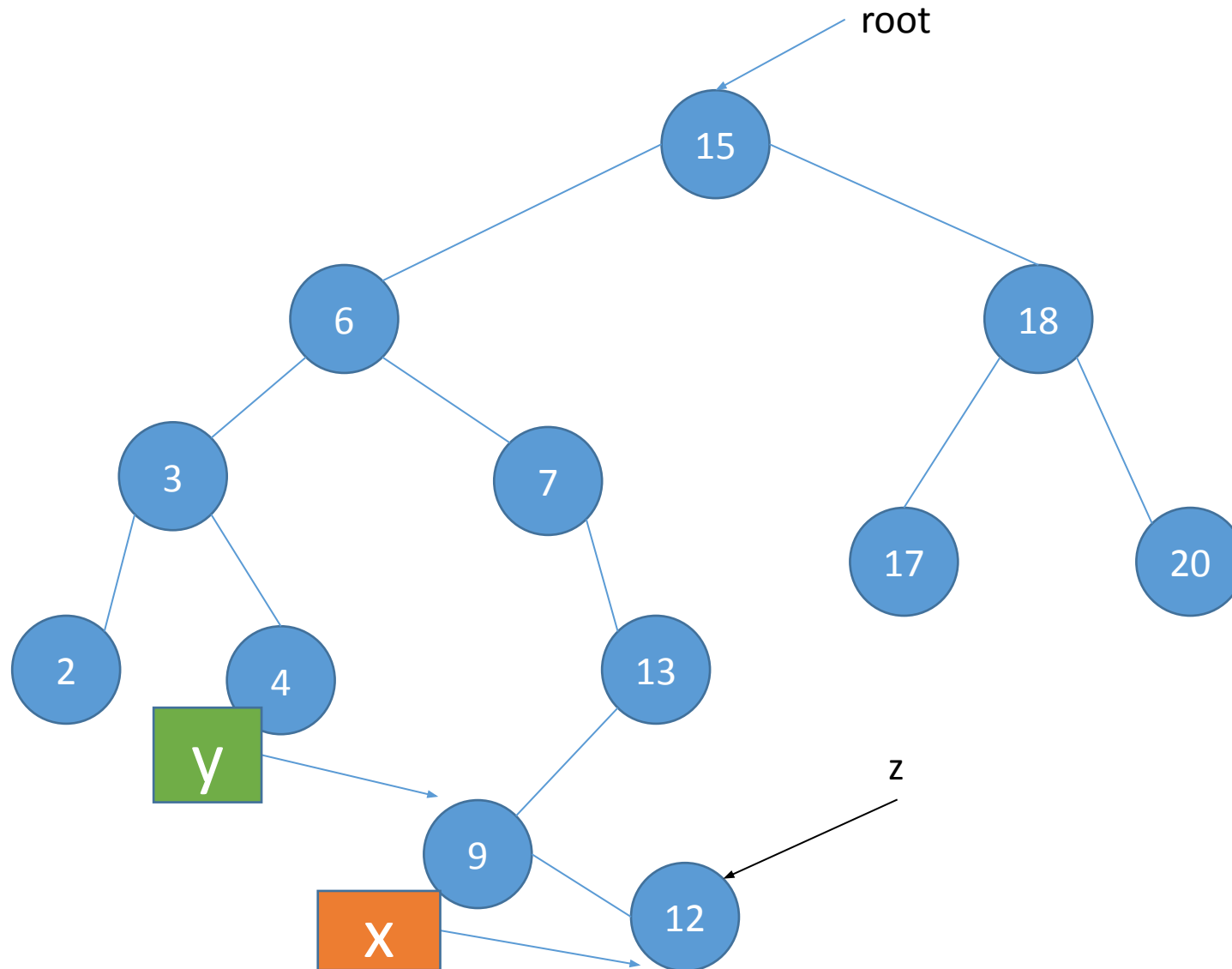
- Let us insert 12
 - We need to find the **position** for the new Tree Node with the value 12 **X**
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (i.e it is **Y** connected)

Insert Function



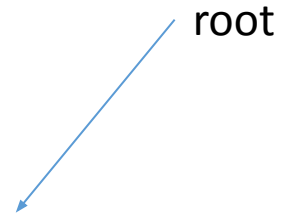
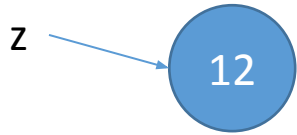
- Let us insert 12
 - We need to find the **position** for the new Tree Node with the value 12 **X**
 - We need to make sure that it points to the parent Tree Node and the parent Tree node points to it (i.e it is **Y** connected)

Insert Function



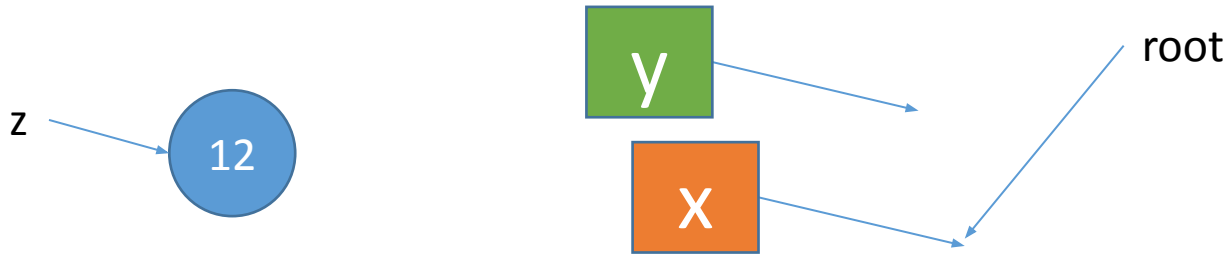
- Some observations:
 - The search goes on until x becomes **NULL**
 - y helps to refer to the **parent**
 - y helps in making the necessary **connections**
 - (y.right) and (x.parent) in this **case**

Insert Function (Empty Tree)



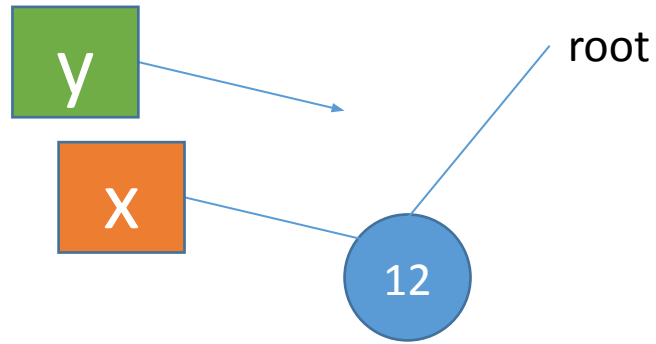
- Some observations:
 - The search goes on until x becomes **NULL**
 - y helps to refer to the **parent**
 - y helps in making the necessary **connections**
 - ~~$(y.right)$ and $(x.parent)$ in this case~~

Insert Function (Empty Tree)

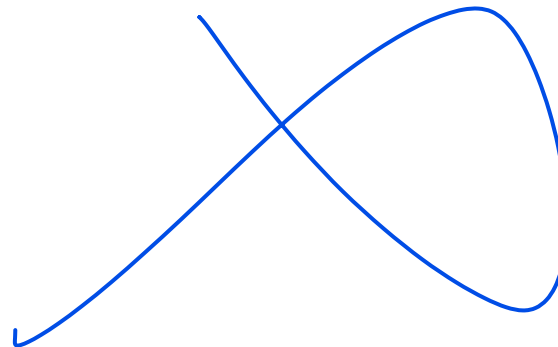


- Some observations:
 - The search goes on until x becomes **NULL**
 - y helps to refer to the **parent**
 - y helps in making the necessary **connections**
 - (y.right) and (x.parent) in this **case**

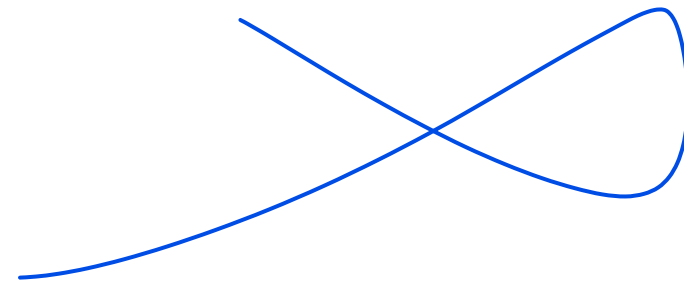
Insert Function (Empty Tree)



- Some observations:
 - The search goes on until x becomes **NULL**
 - y helps to refer to the **parent**
 - y helps in making the necessary **connections**
 - (y.right) and (x.parent) in this **case**

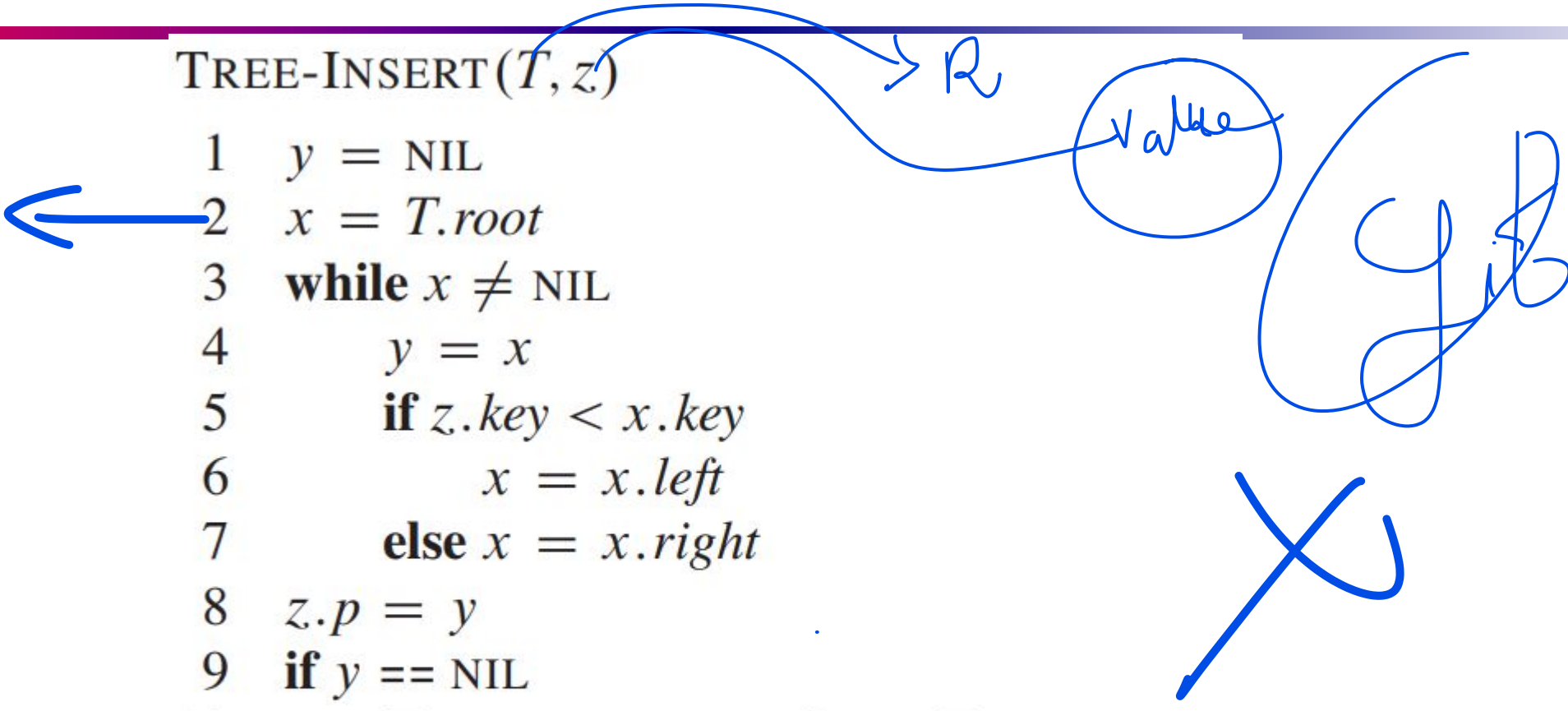


Let us Write the Pseudocode



Let us Write the Pseudocode

TREE-INSERT(T, z)

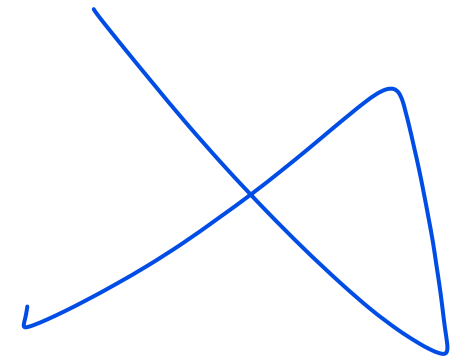
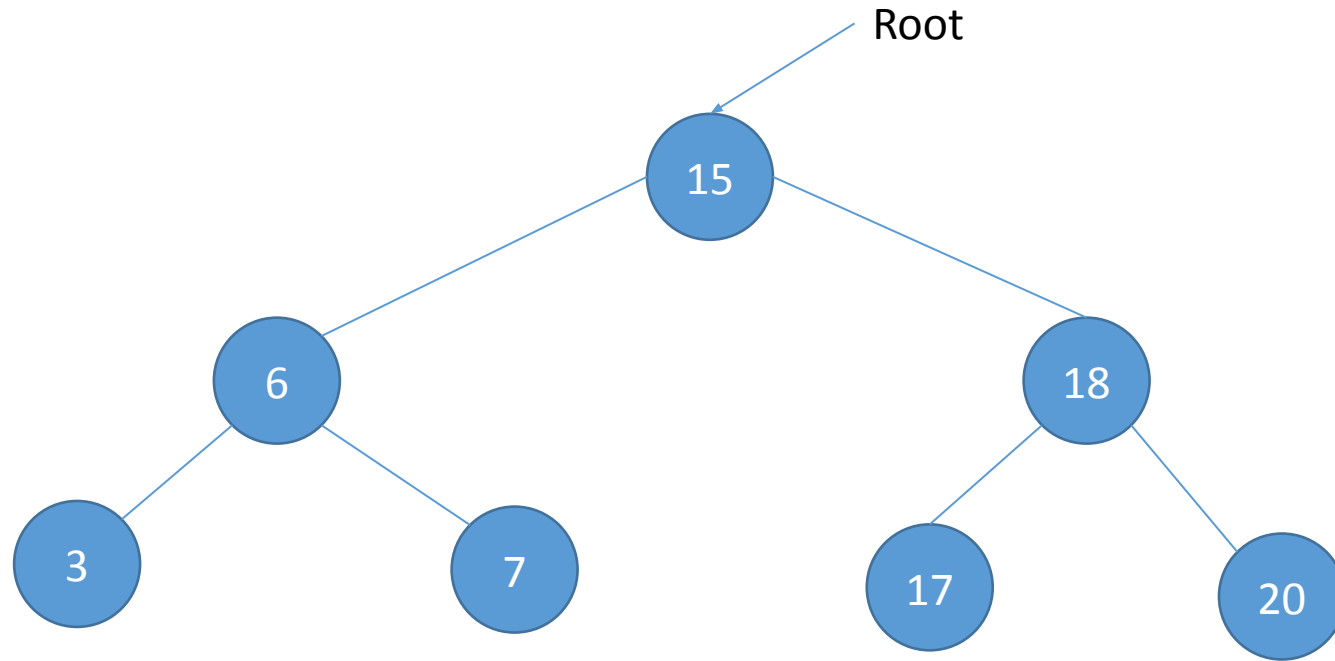


```
1   $y = \text{NIL}$ 
2   $x = T.\text{root}$ 
3  while  $x \neq \text{NIL}$ 
4       $y = x$ 
5      if  $z.\text{key} < x.\text{key}$ 
6           $x = x.\text{left}$ 
7      else  $x = x.\text{right}$ 
8   $z.p = y$ 
9  if  $y == \text{NIL}$ 
10      $T.\text{root} = z$       // tree  $T$  was empty
11  elseif  $z.\text{key} < y.\text{key}$ 
12      $y.\text{left} = z$ 
13  else  $y.\text{right} = z$ 
```

Deletion

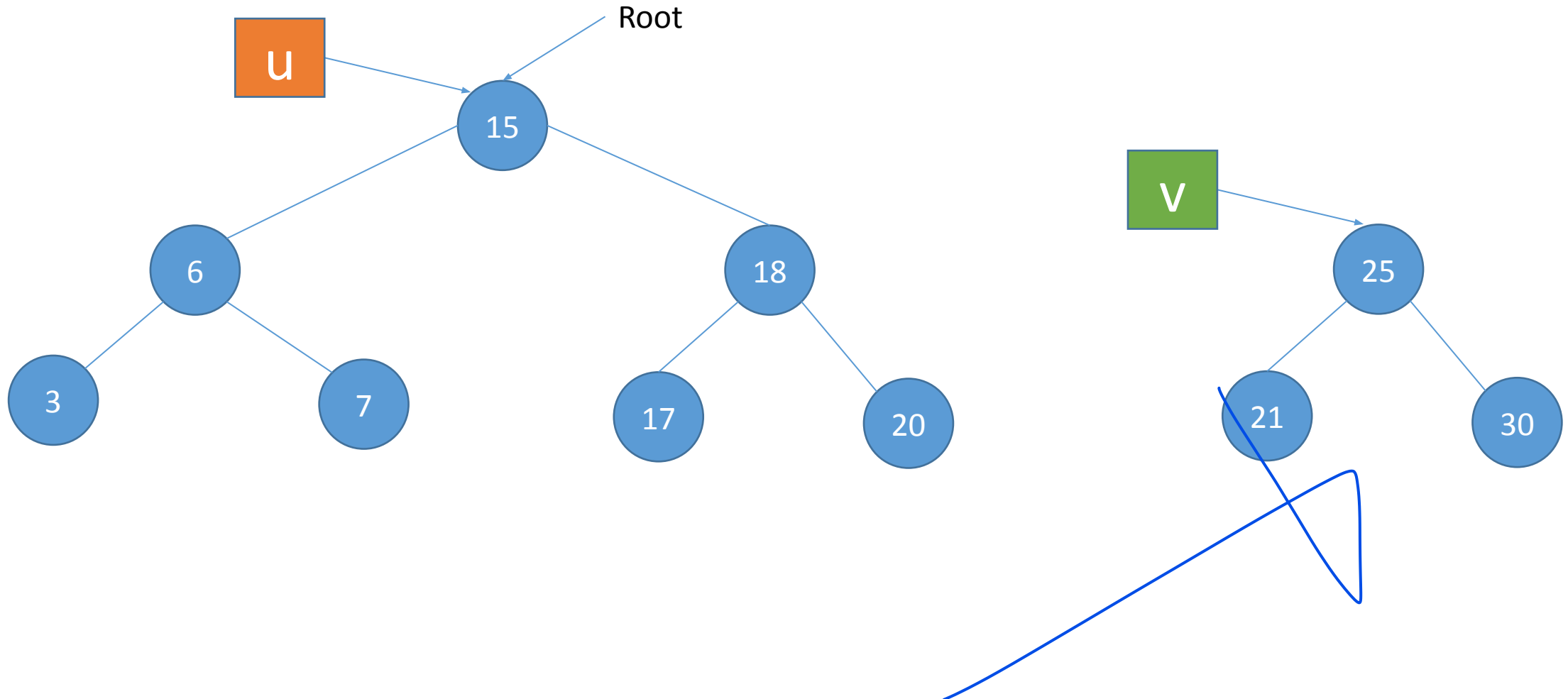
- Before learning about deletion we need to know about another function
 - Transplant
 - replaces one subtree as a child of its parent with another subtree.\
- Transplant (u,v)
 - u – node u is the parent of a subtree which will be replaced
 - v – node b is the parent of a subtree which replace the subtree rooted at u

Transplant



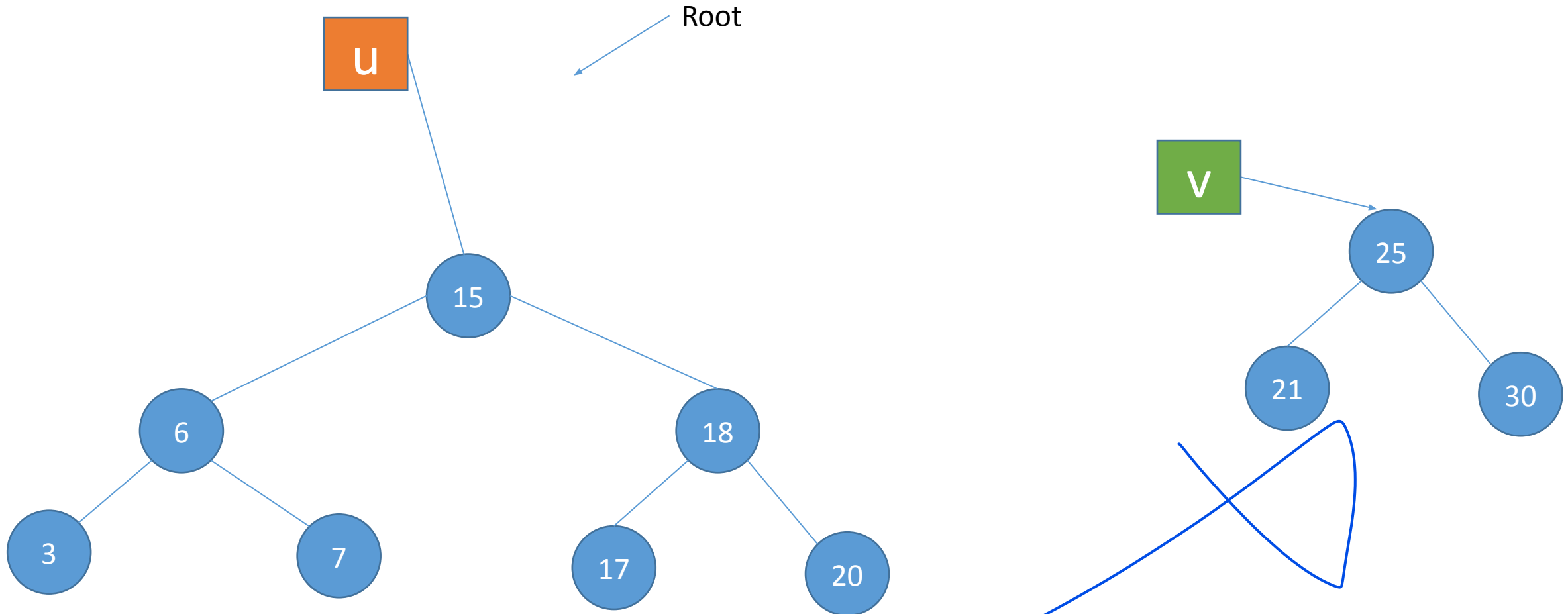
Transplant

Case 1: u is the root



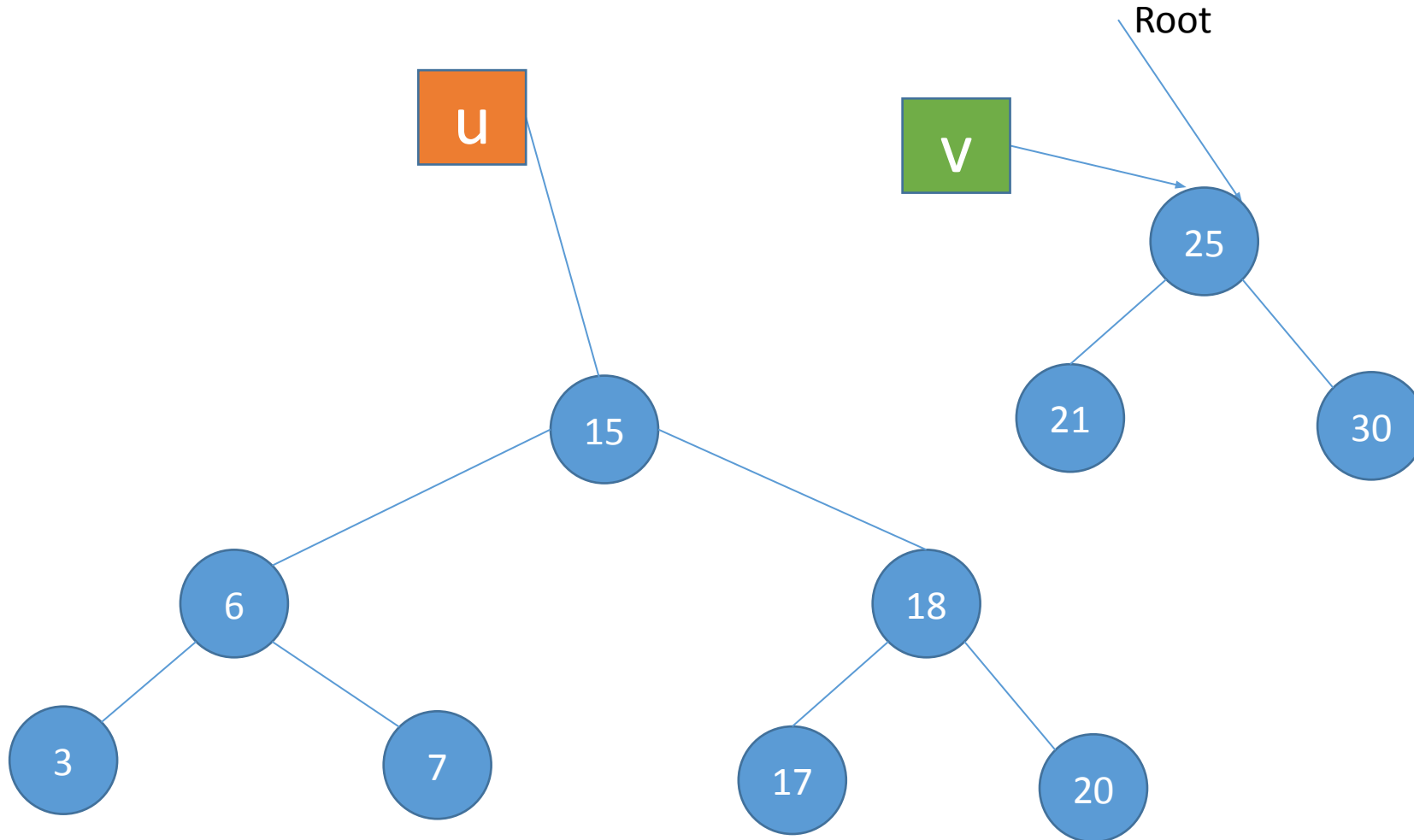
Transplant

Case 1: u is the root



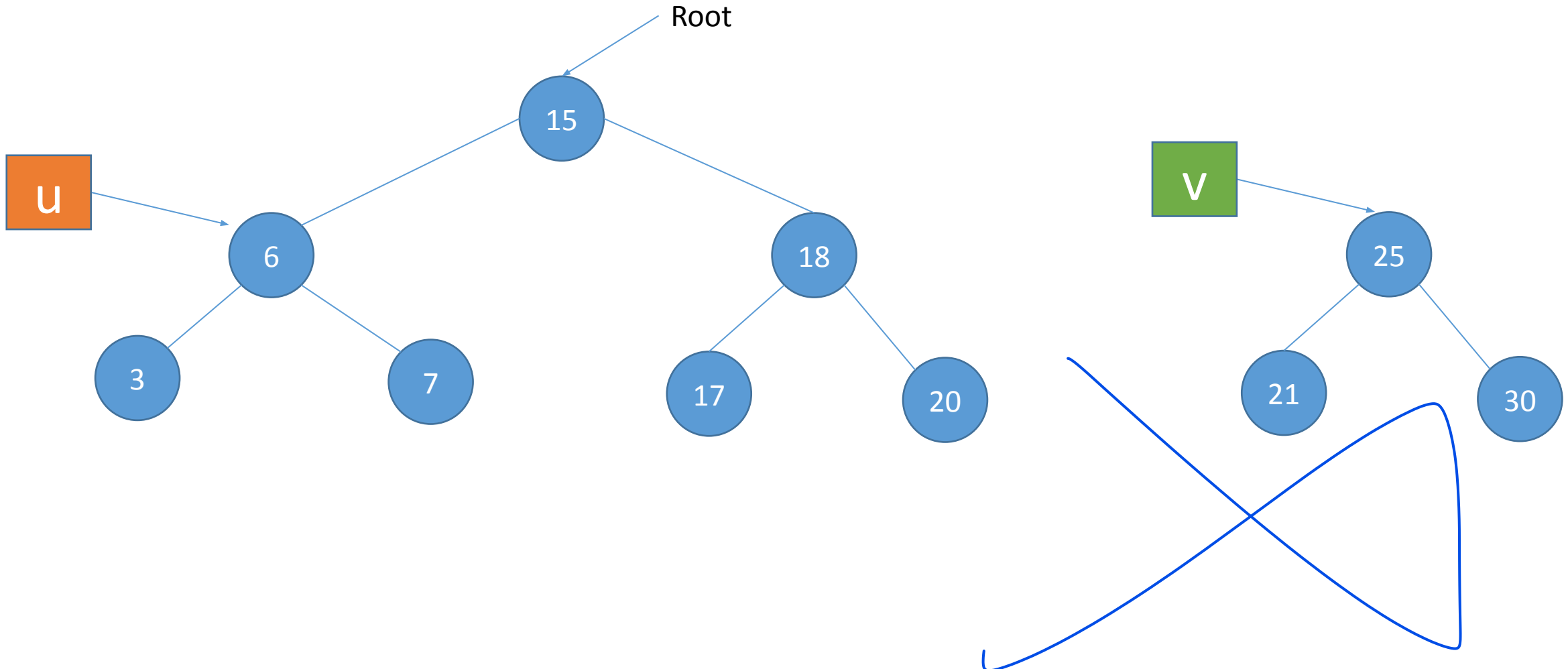
Transplant

Case 1: u is the root



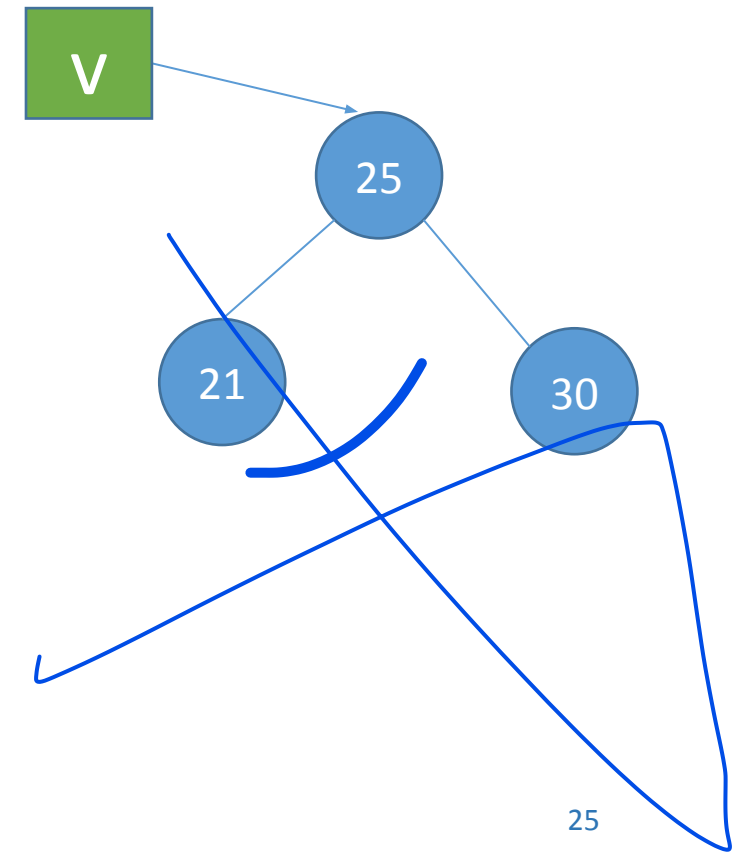
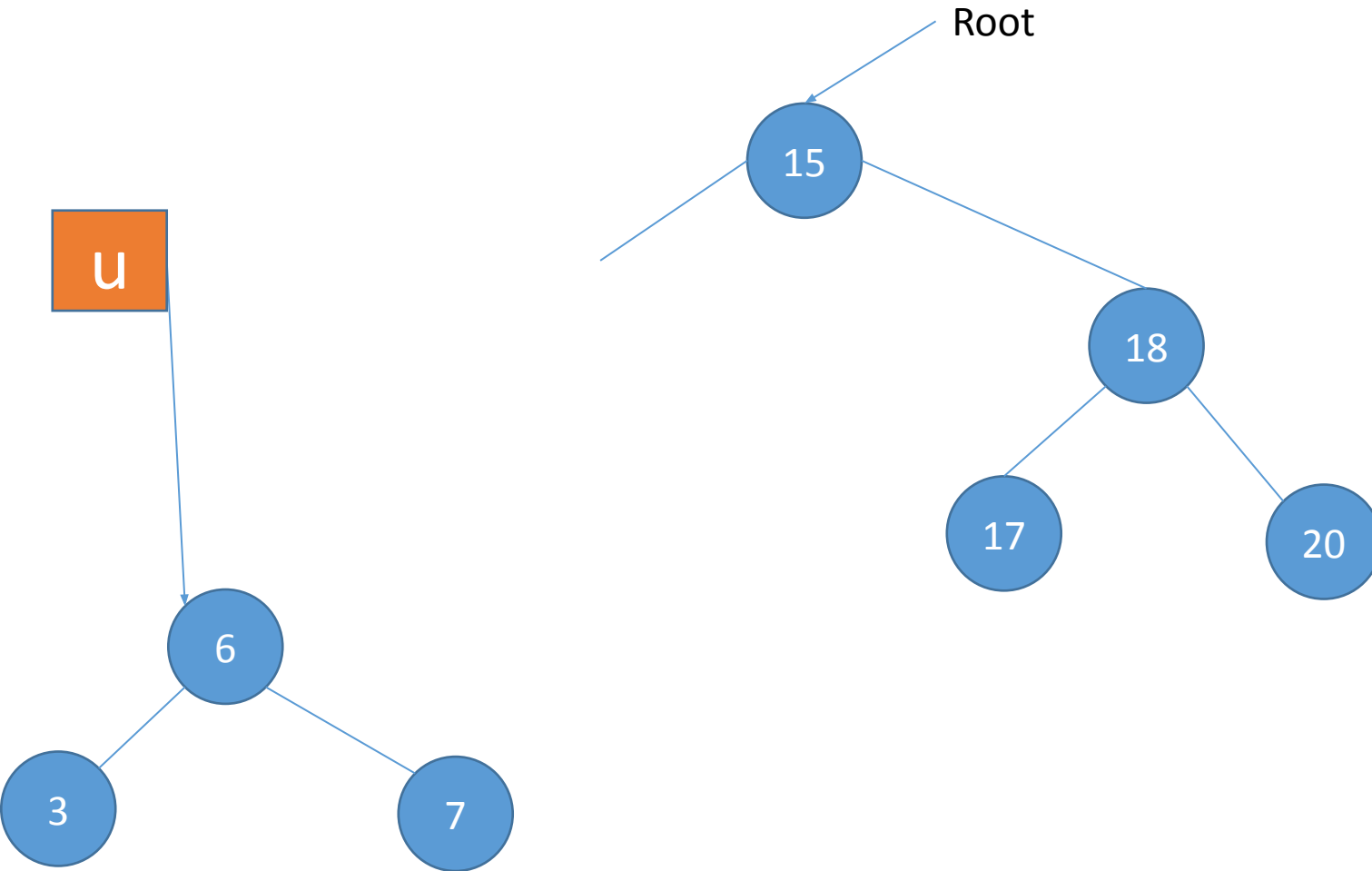
Transplant

Case 2: u is a left child



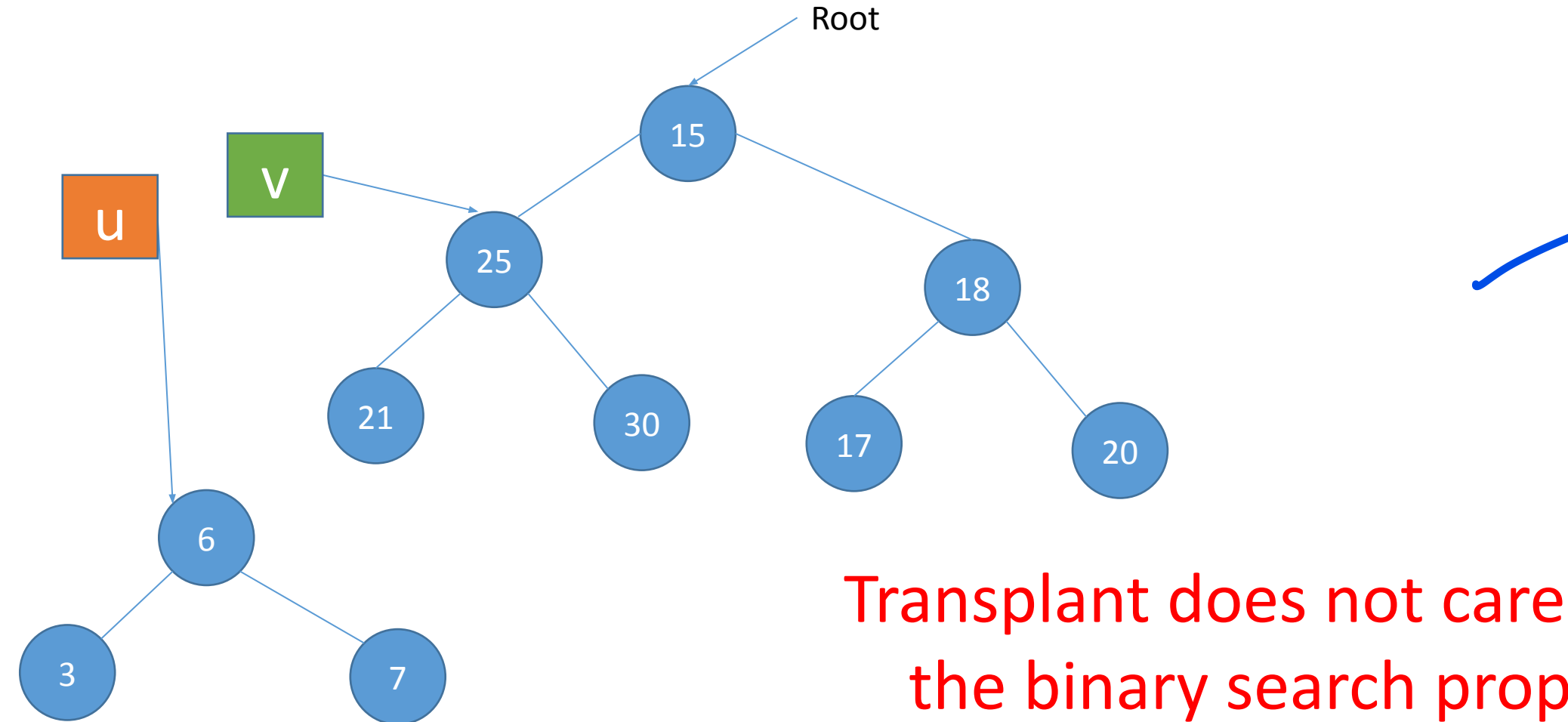
Transplant

Case 2: u is a left child



Transplant

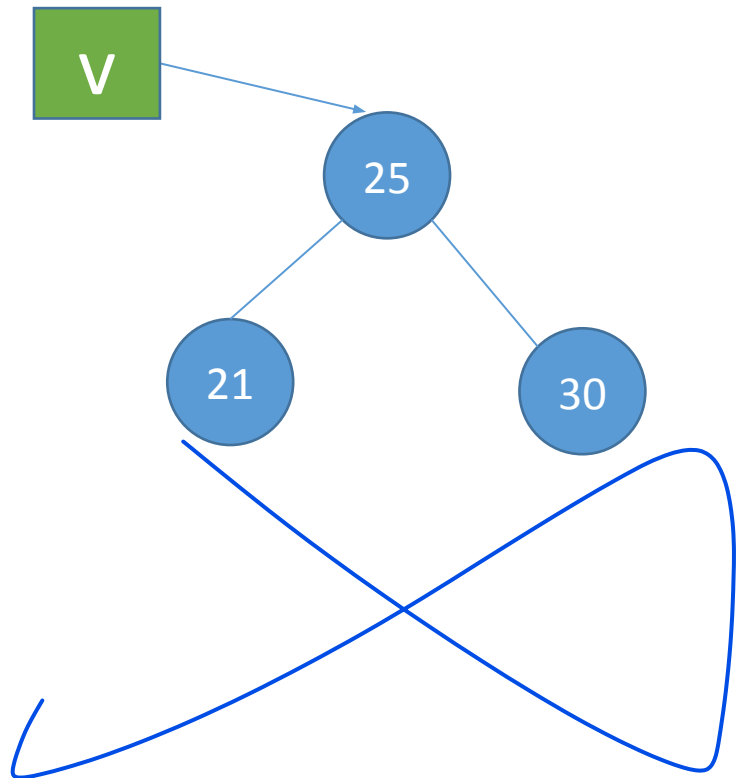
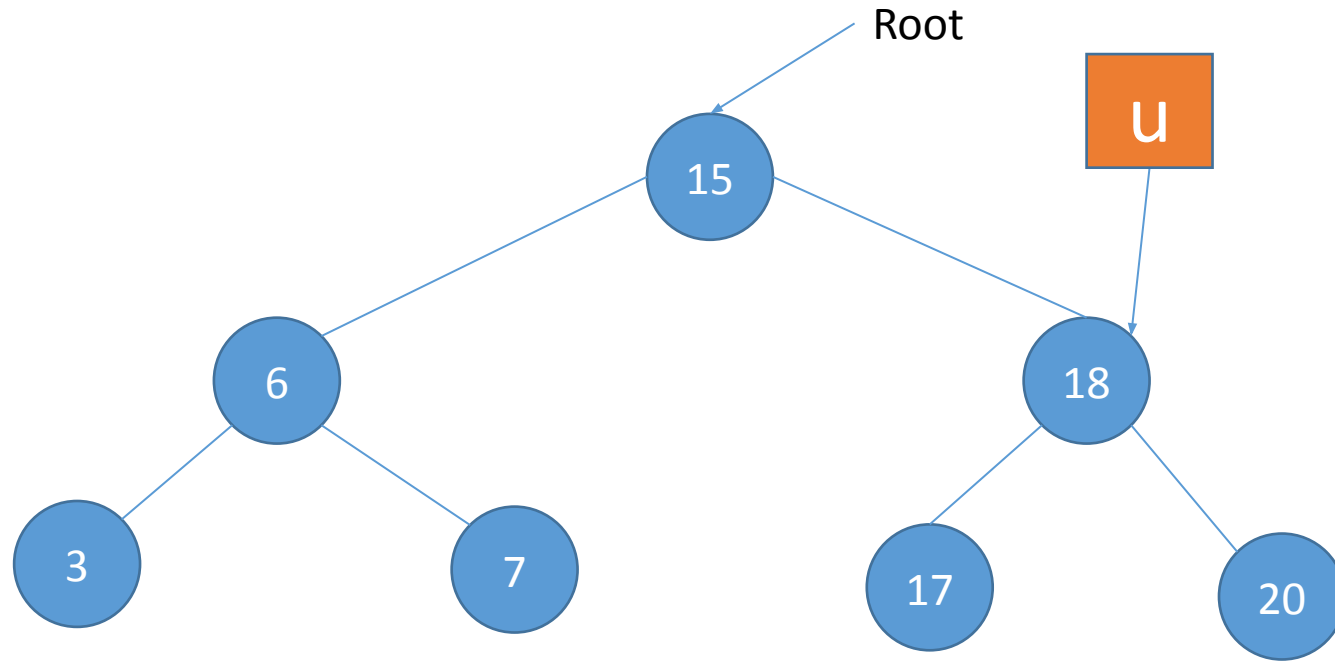
Case 2: u is a left child



Transplant does not care about
the binary search property

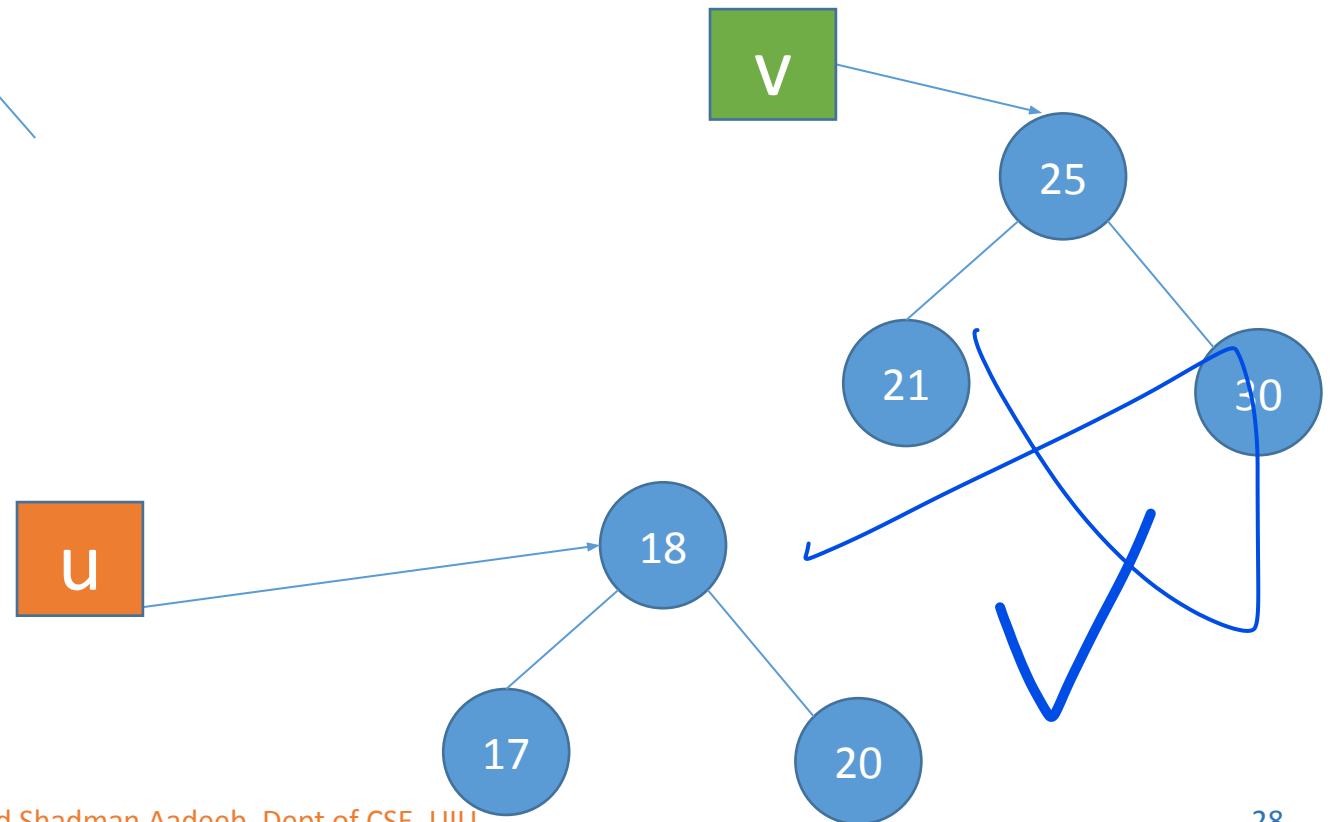
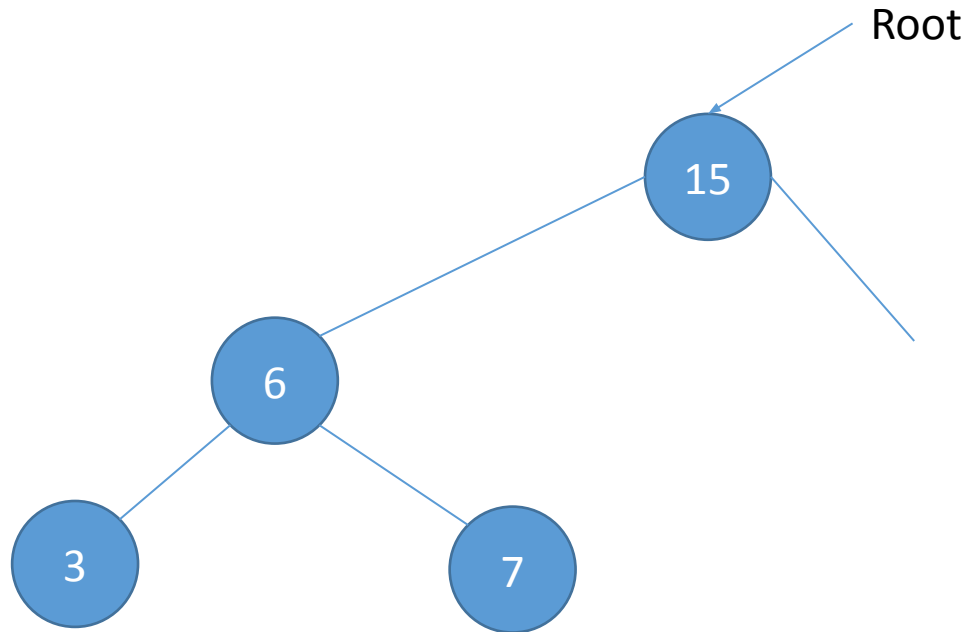
Transplant

Case 3: u is a right child



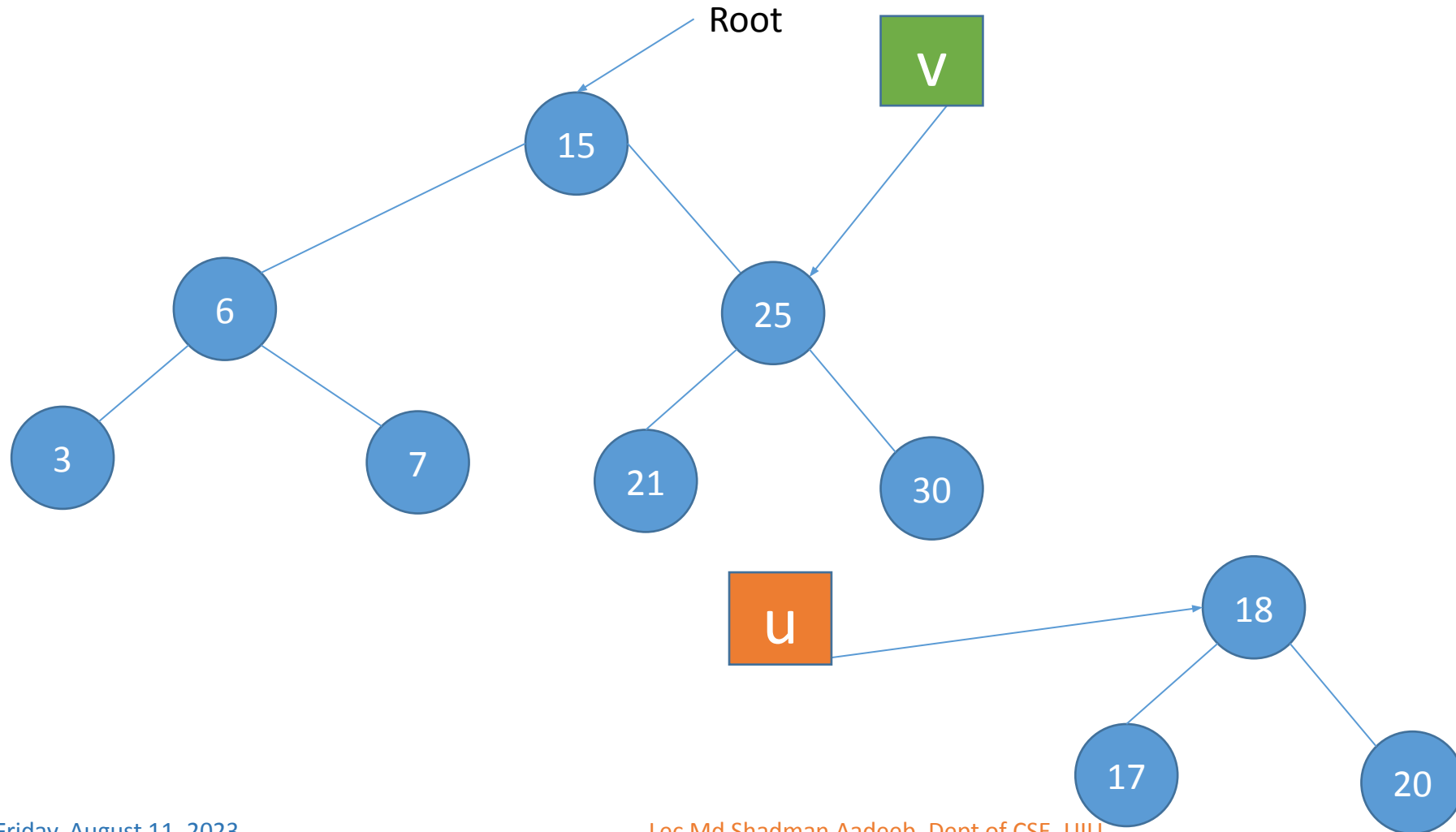
Transplant

Case 3: u is a right child



Transplant

Case 3: u is a right child

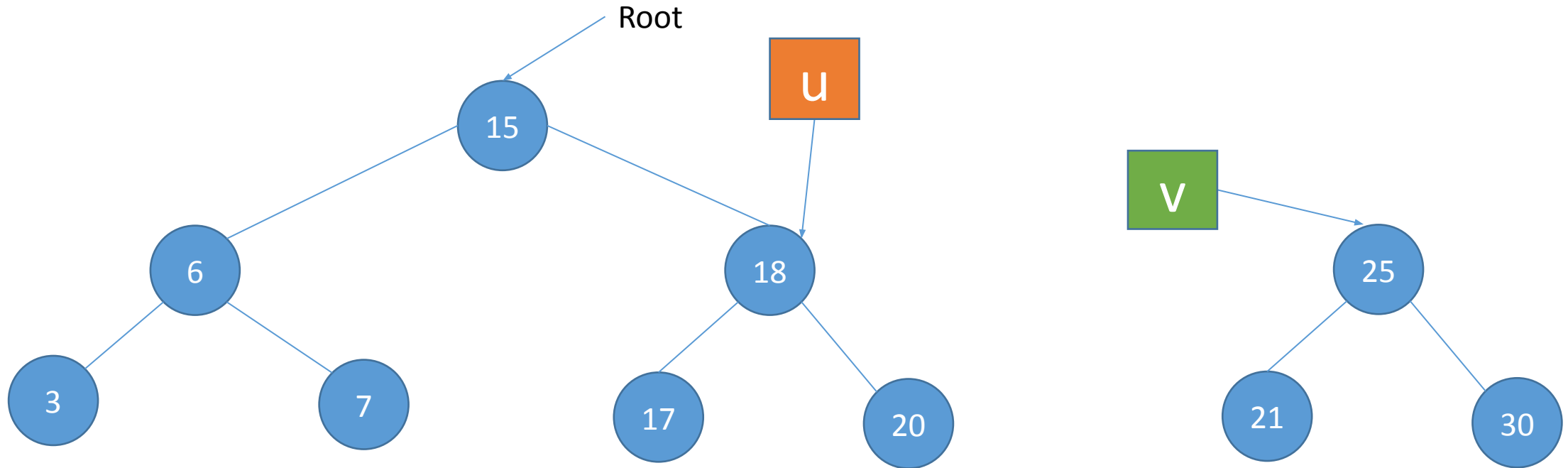


Transplant

- The replaced child keeps on pointing to the parent.
- This does not cause any issue.
- Because any **pointer outside the tree** can point to any part of a tree.
- It is okay, as long as the **pointers inside the tree** do not point some where else.
- Let us observe case 3 again

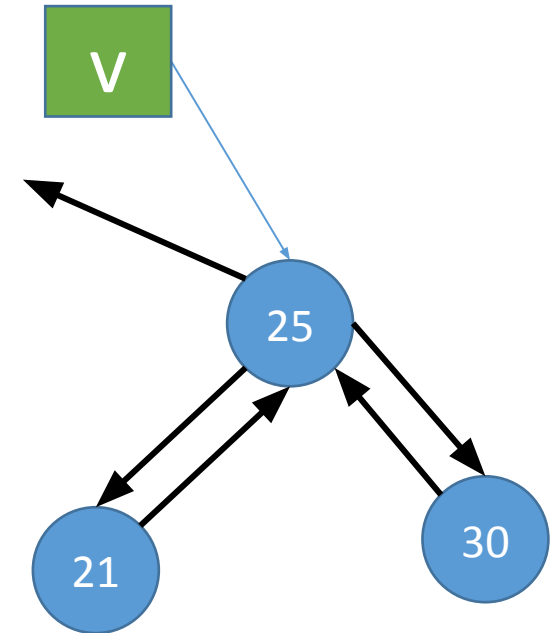
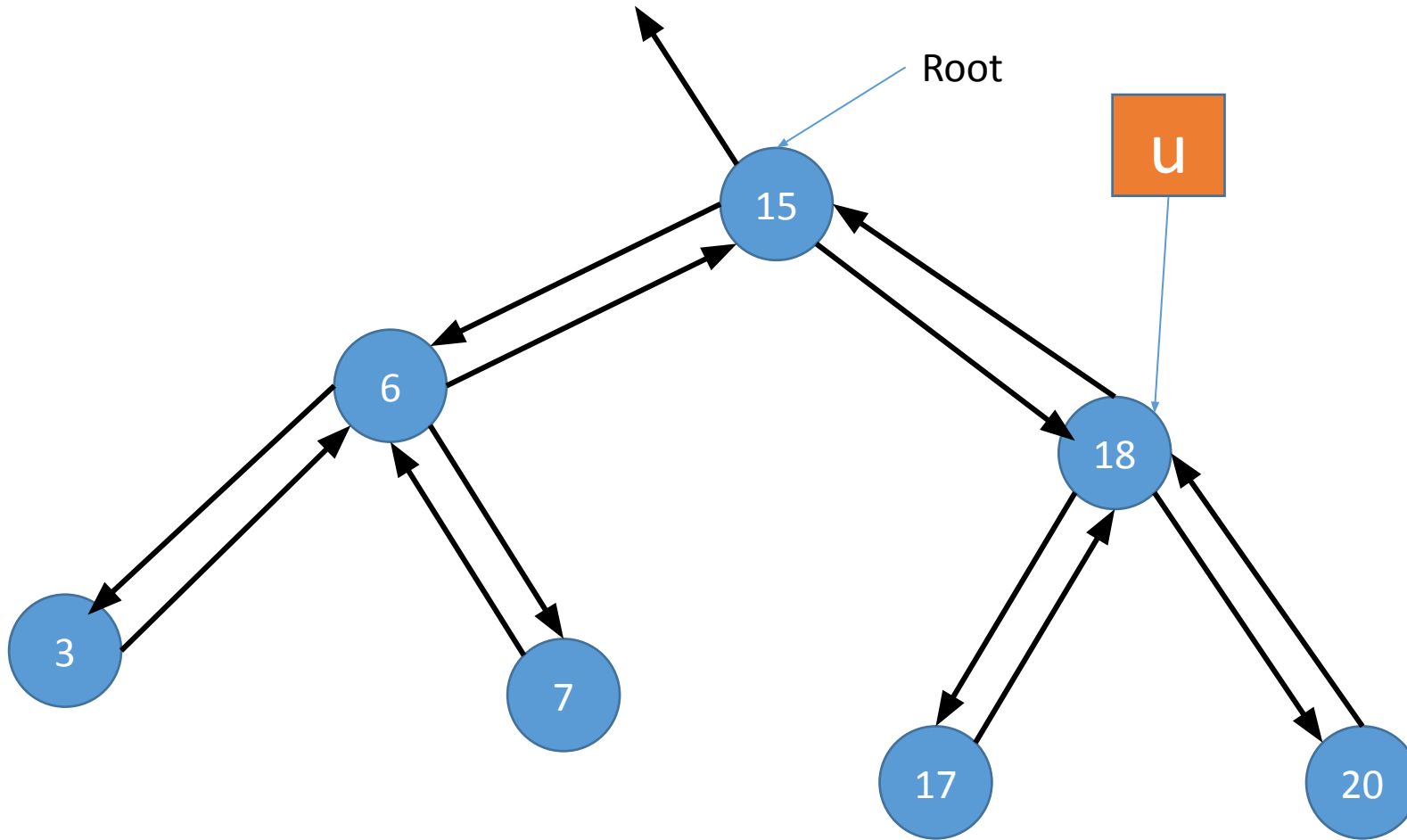
Transplant

Case 3: u is a right child



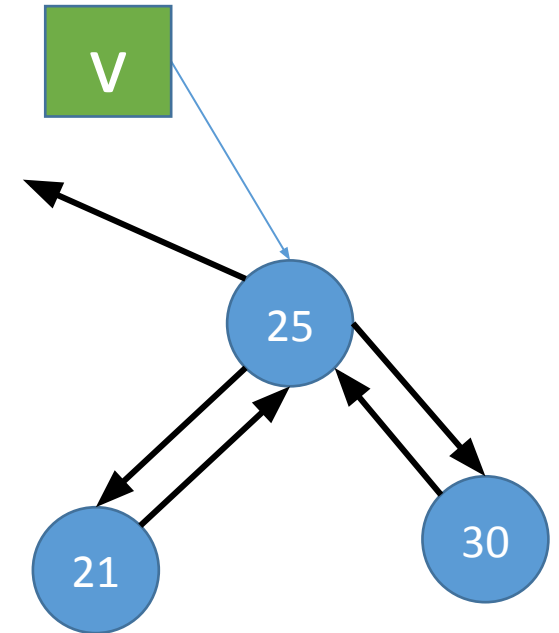
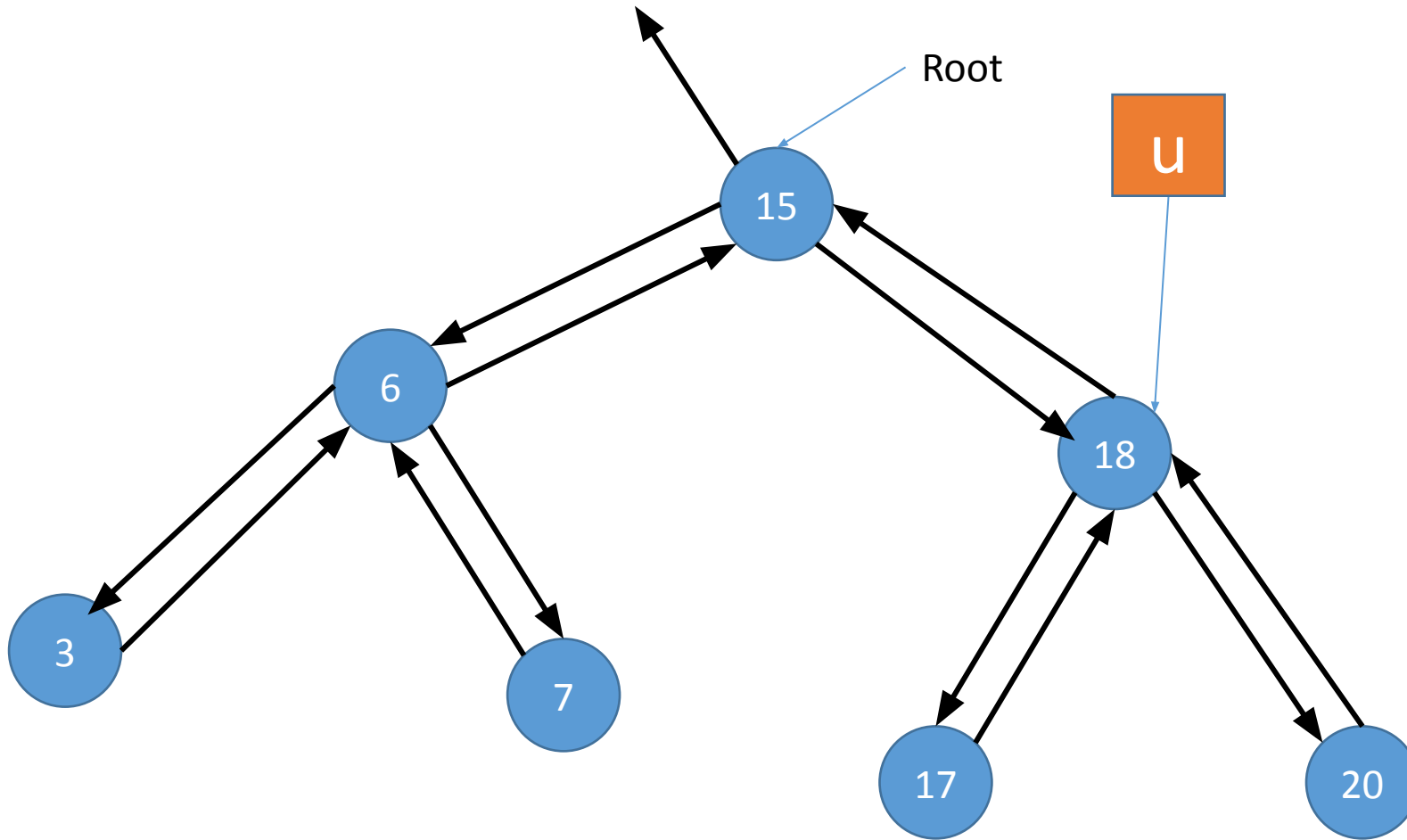
Transplant

Case 3: u is a right child



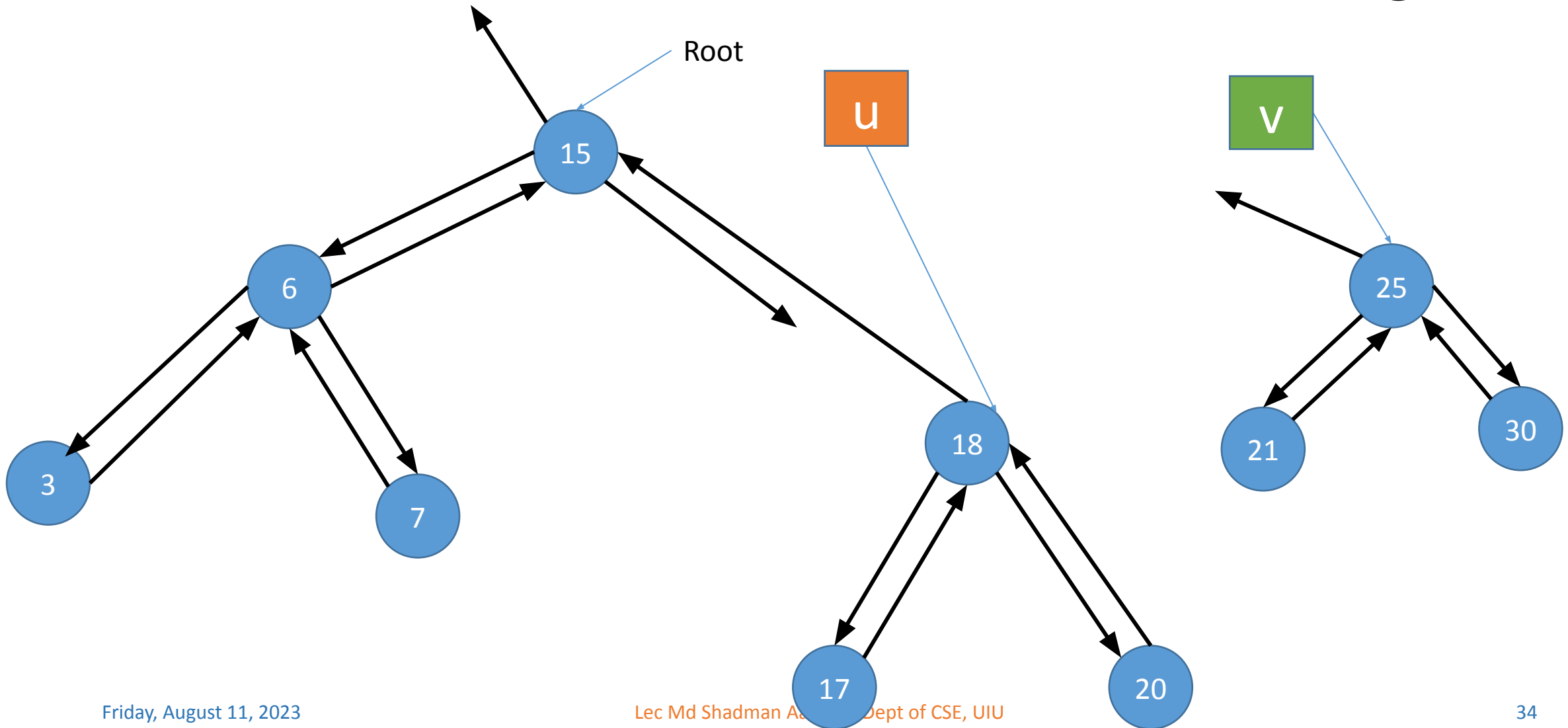
Transplant

Case 3: u is a right child



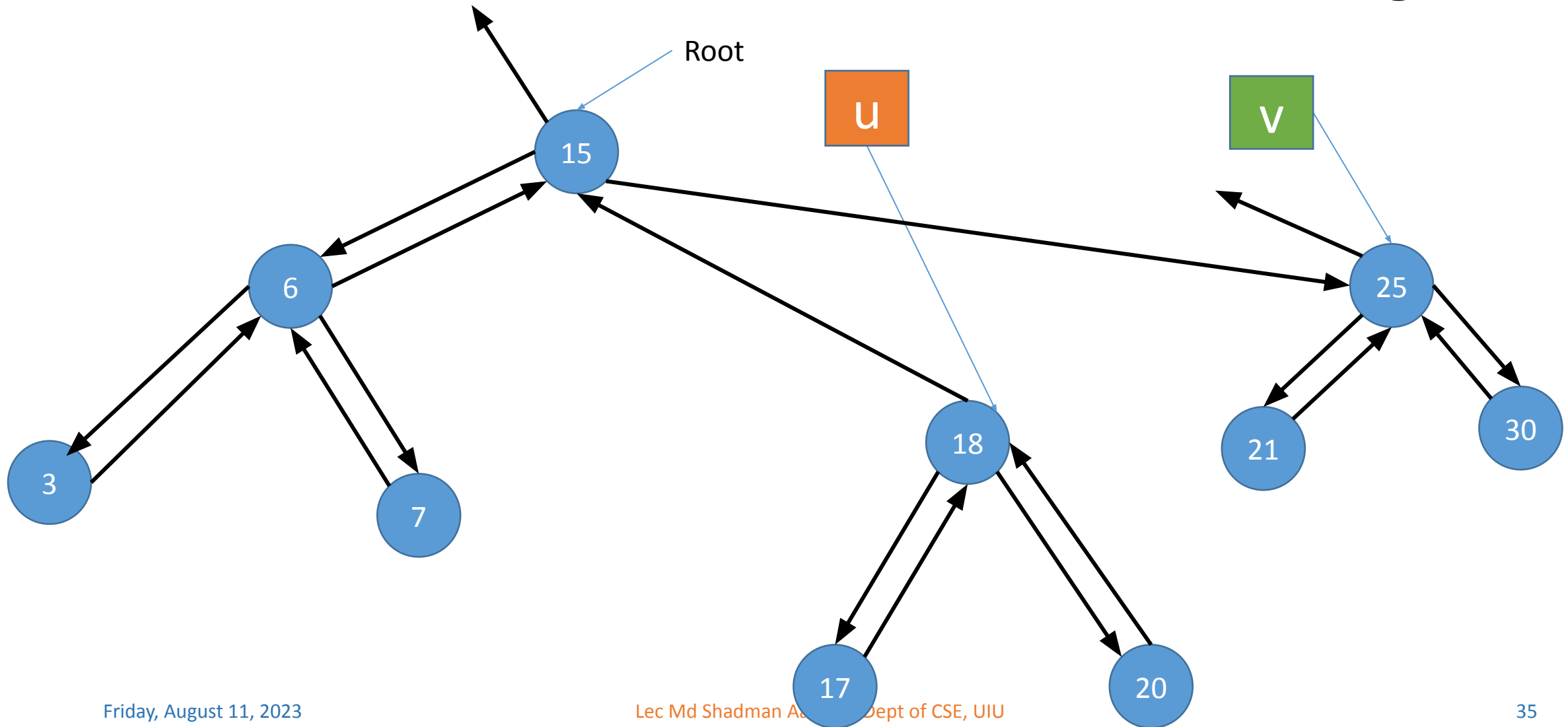
Transplant

Case 3: u is a right child



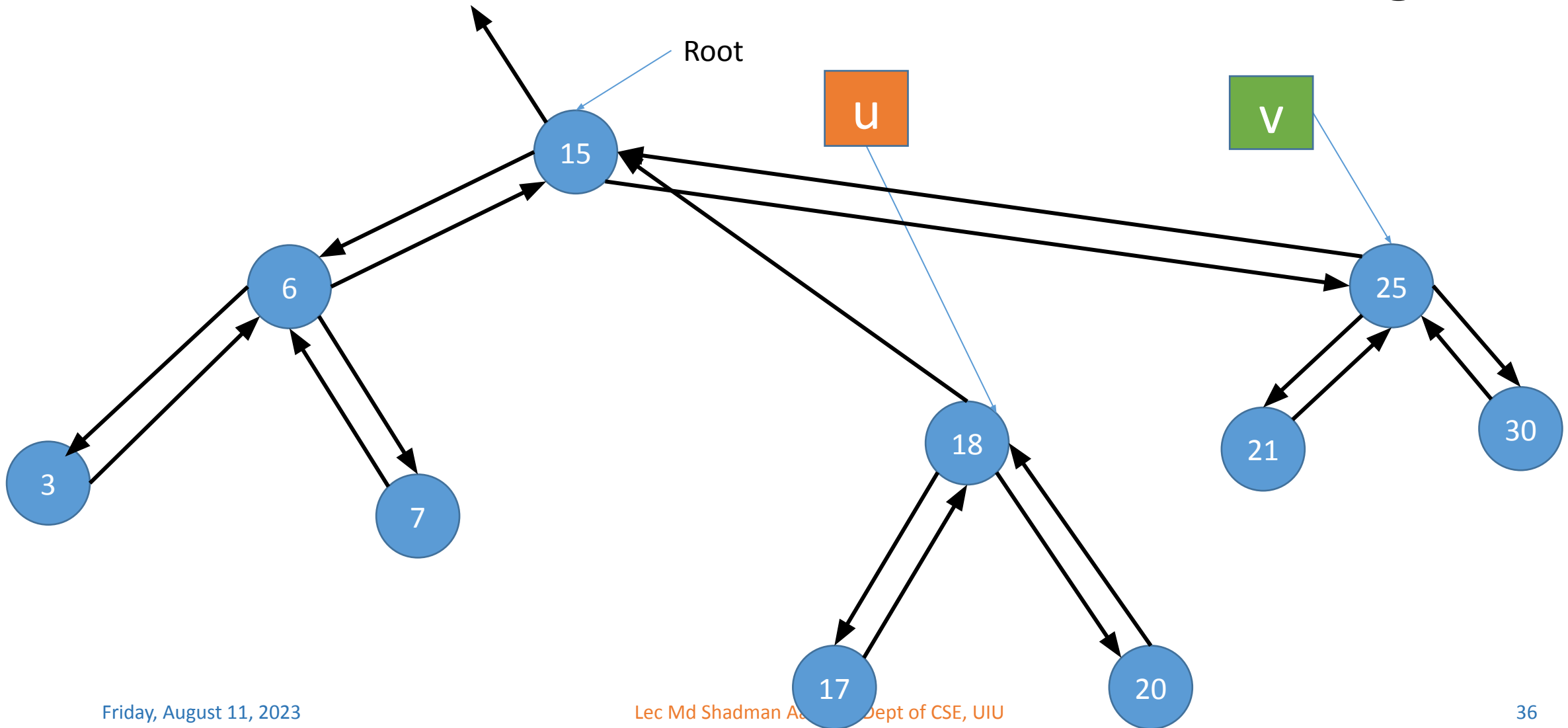
Transplant

Case 3: u is a right child

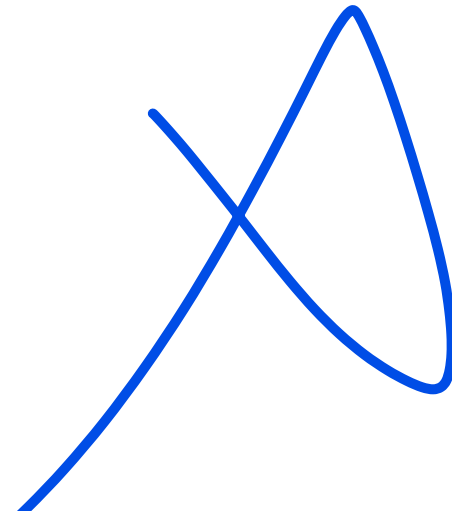


Transplant

Case 3: u is a right child



Write the Pseudocode for Transplant



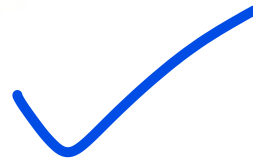
Write the Pseudocode for Transplant

TRANSPLANT(T, u, v)

```
1  if  $u.p == \text{NIL}$ 
2       $T.root = v$ 
3  elseif  $u == u.p.left$ 
4       $u.p.left = v$ 
5  else  $u.p.right = v$ 
6  if  $v \neq \text{NIL}$ 
7       $v.p = u.p$ 
```

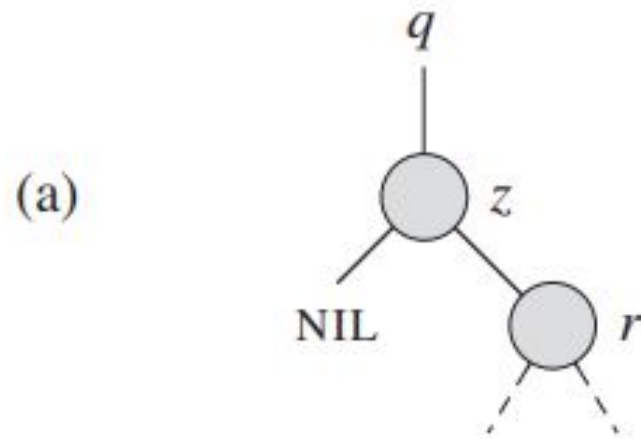
Null

just



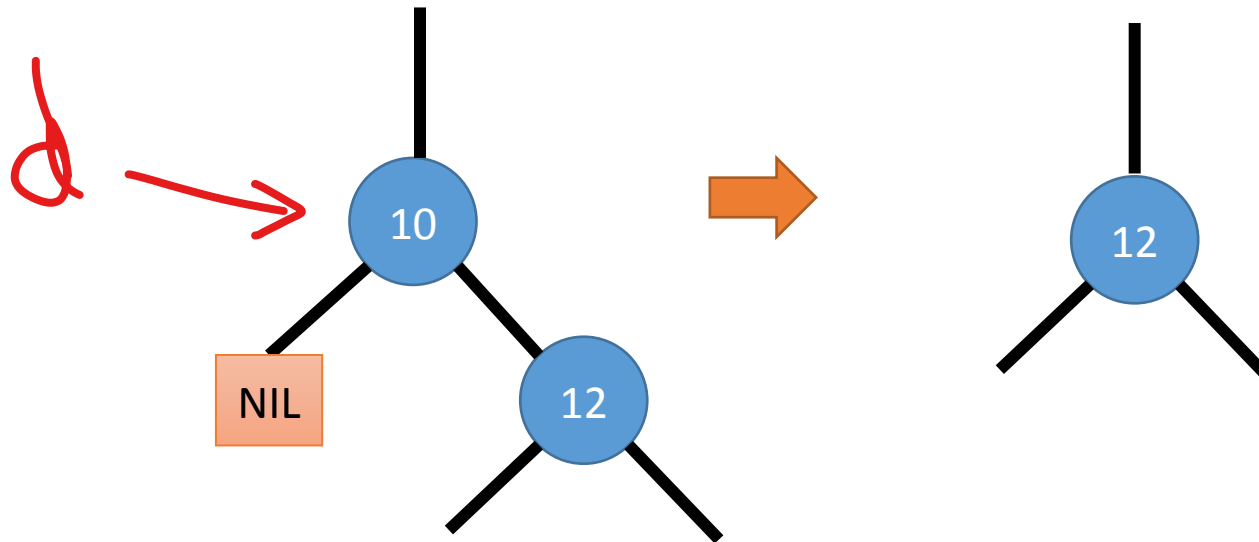
Deletion

- Let us observe the following four cases:
- Case 1: Node z (**node to be deleted**) has no left child



Deletion

- Let us observe the following four cases:
- Case 1: Node z (node to be deleted) has no left child**

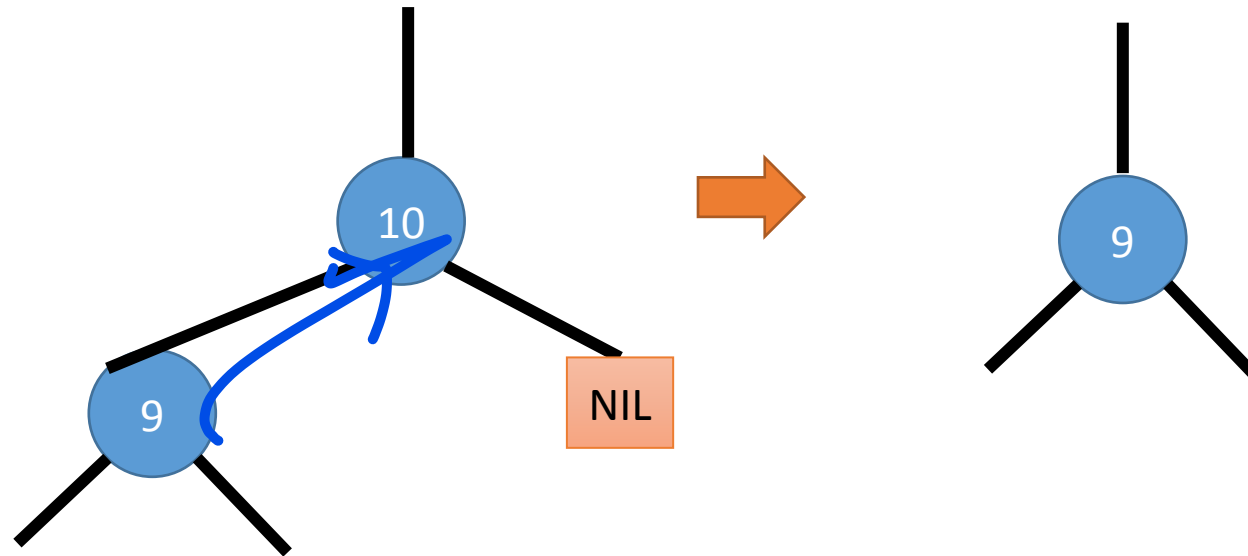


A simple transplant operation can get the task done

Transplant (z, z.right)

Deletion

- Let us observe the following four cases:
- Case 2: Node z (node to be deleted)** has no right child



A simple transplant operation can get the task done
 $\text{Transplat}(z, z.\text{left})$

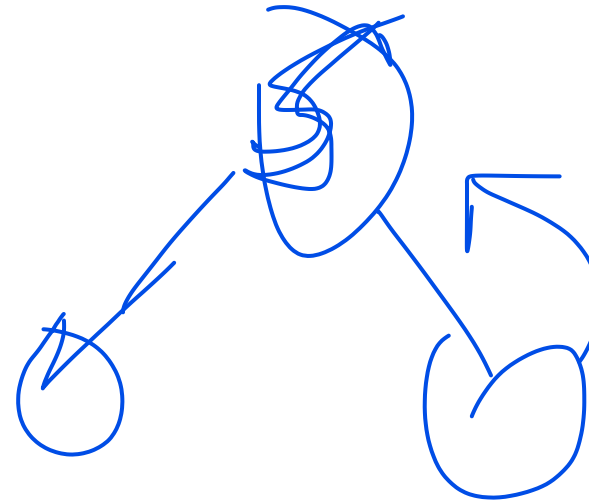
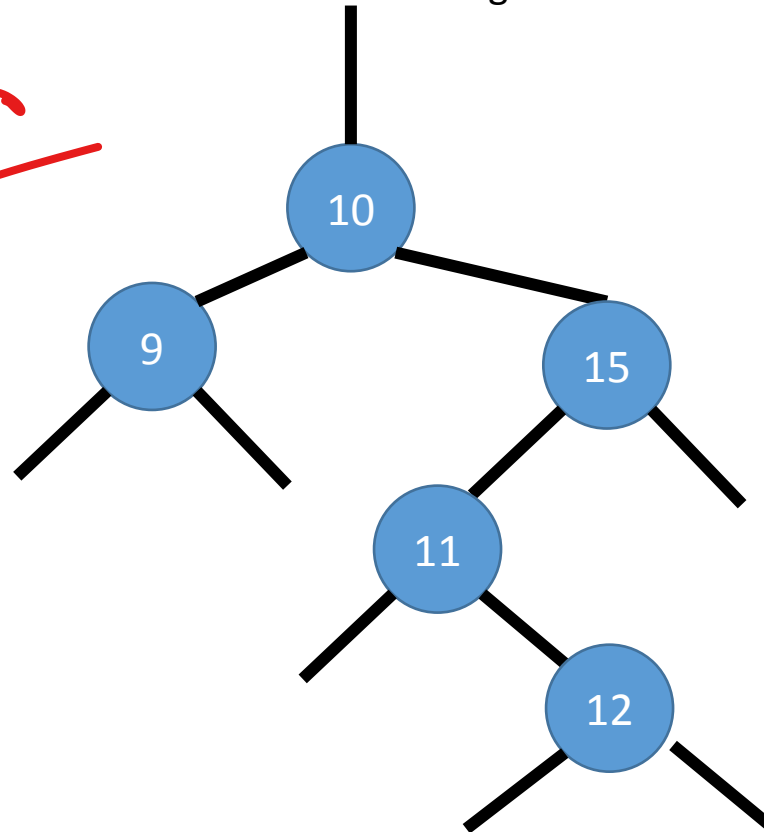
Deletion

- Let us observe the following four cases:

- Case 3:

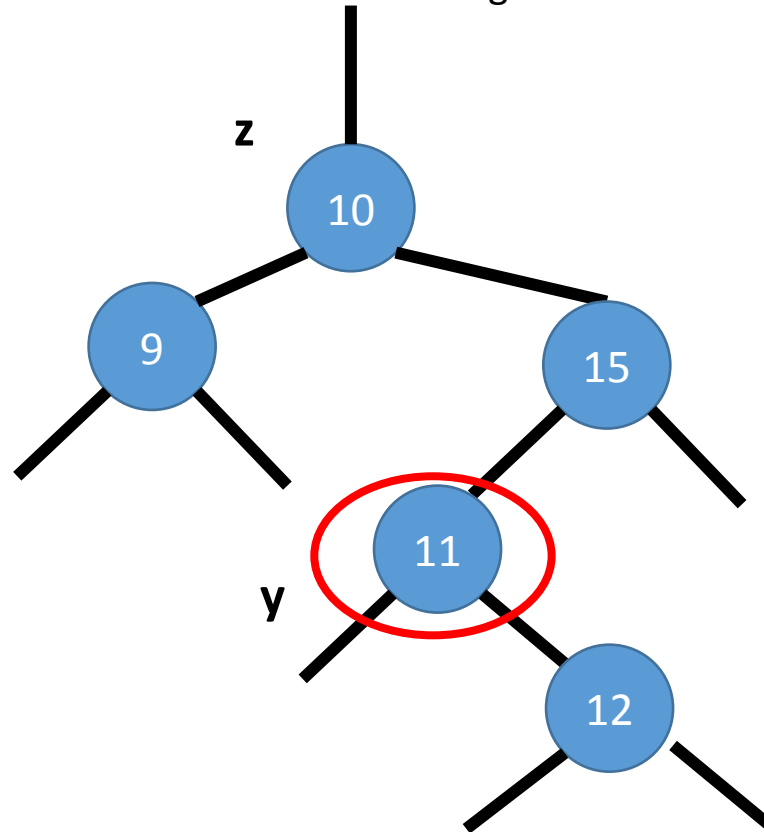
- Node z (**node to be deleted**) has both children
- We need to find the minimum child on the right sub tree and the minimum value is **not the immediate child**

Success



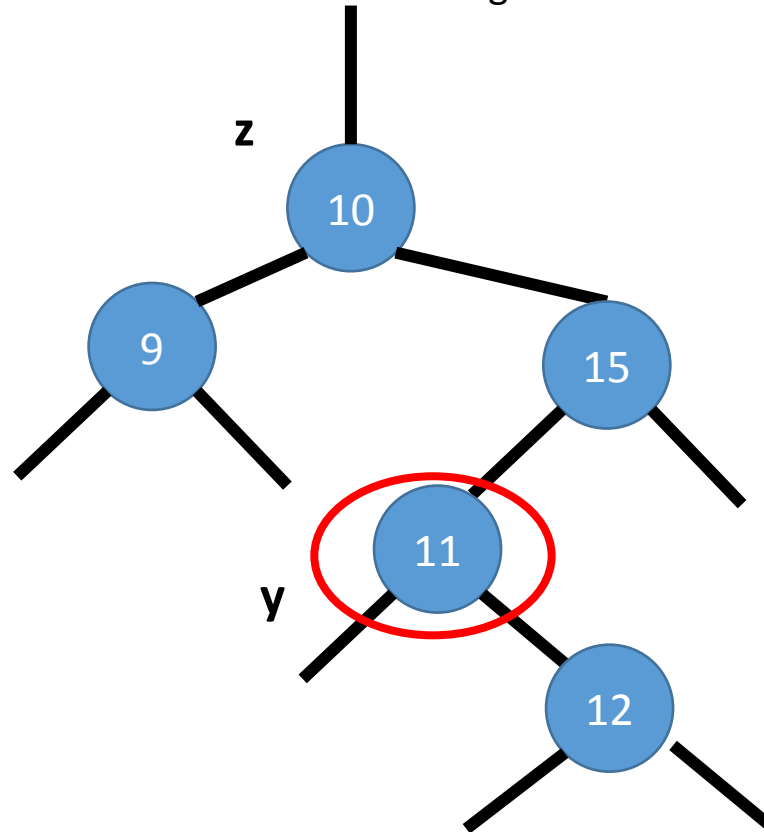
Deletion

- Let us observe the following four cases:
- Case 3:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value is not the immediate child



Deletion

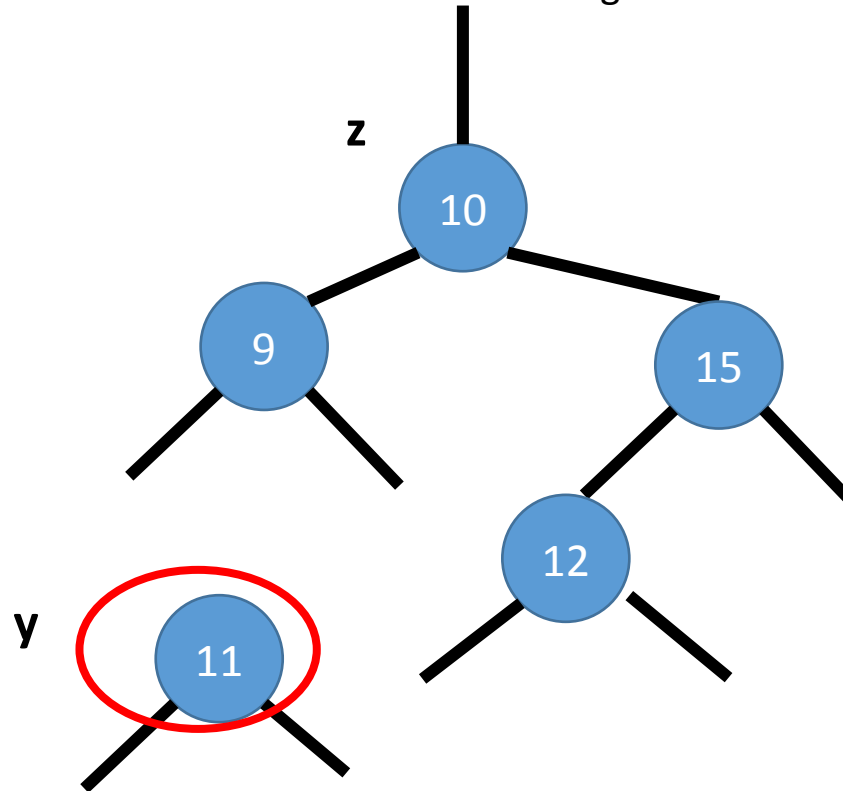
- Let us observe the following four cases:
- Case 3:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value is not the immediate child



Let the minimum node be y
Transplant y with its right child

Deletion

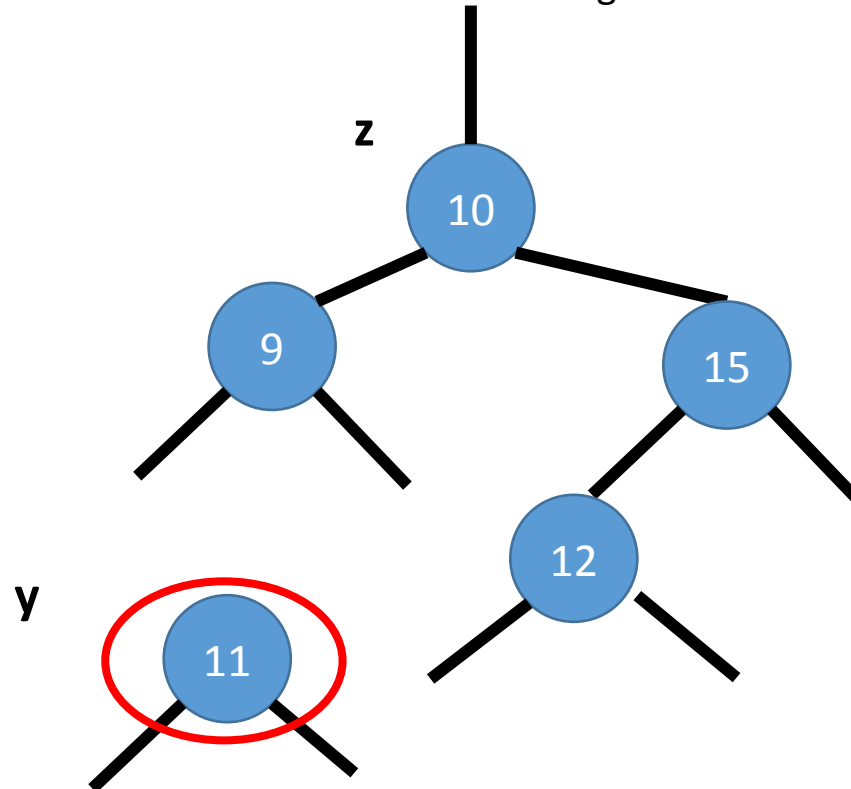
- Let us observe the following four cases:
- Case 3:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value is not the immediate child



Let the minimum node be y
Transplant y with its right child

Deletion

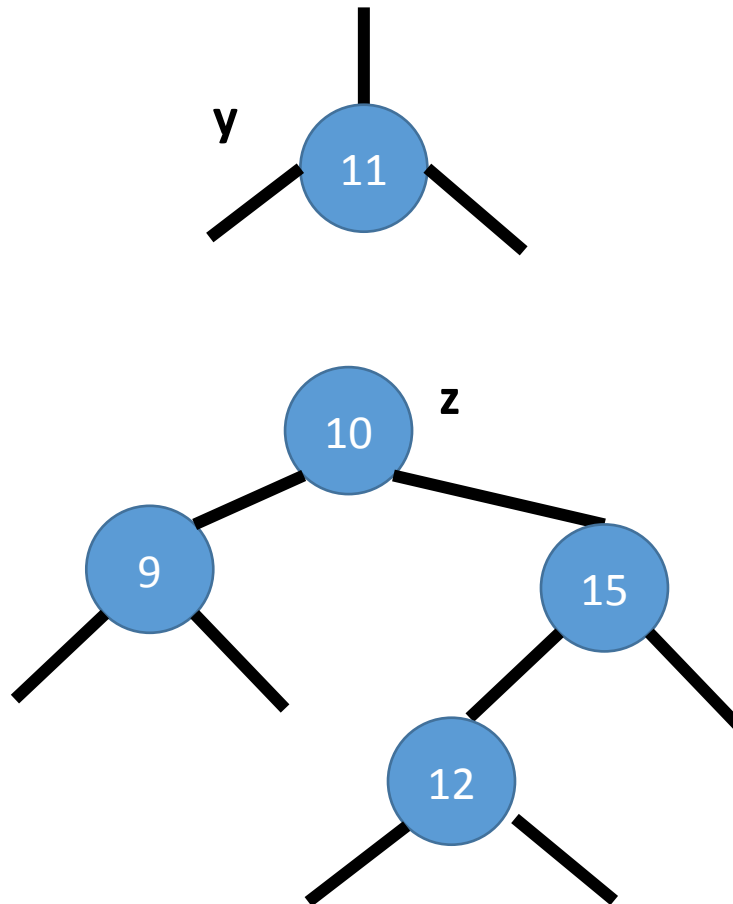
- Let us observe the following four cases:
- Case 3:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value is not the immediate child



Let the minimum node be y
Transplant y with its right child.
Transplant z by y

Deletion

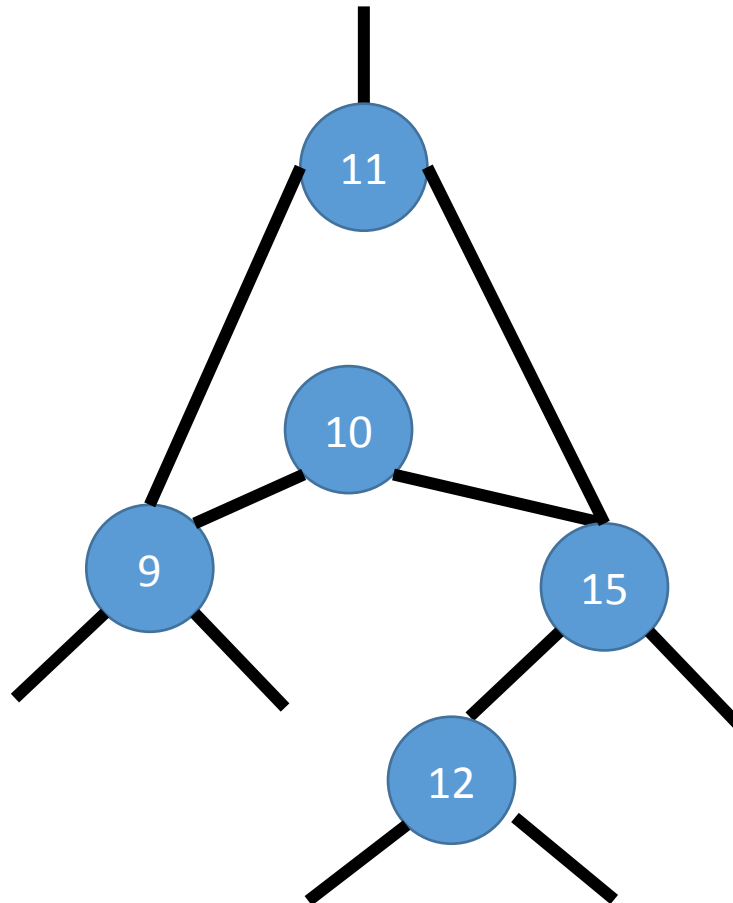
- Let us observe the following four cases:
- Case 3:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value is not the immediate child



Let the minimum node be y
Transplant y with its right child.
Transplant z by y
Make the parent child connections
on both the sides

Deletion

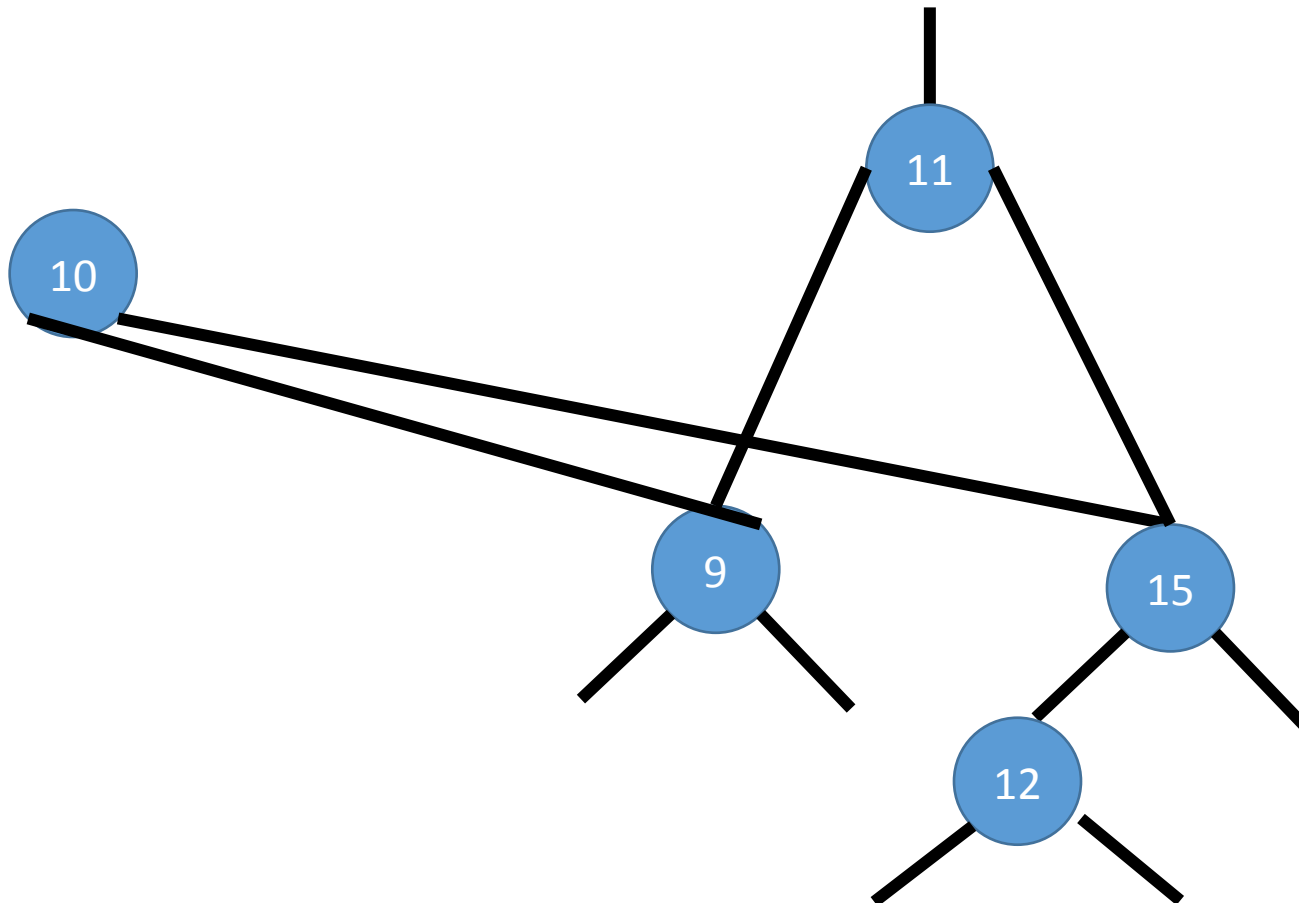
- Let us observe the following four cases:
- Case 3:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value is not the immediate child



Let the minimum node be y
Transplant y with its right child.
Transplant z by y
Make the parent child connections
on both the sides

Deletion

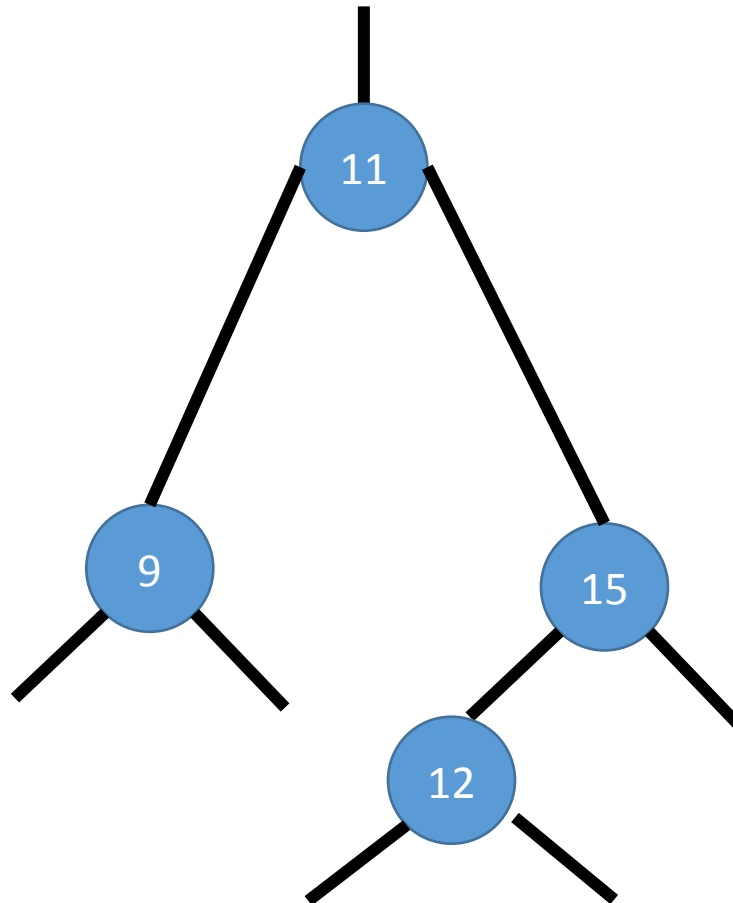
- Let us observe the following four cases:
- Case 3:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value is not the immediate child



Let the minimum node be y
Transplant y with its right child.
Transplant z by y
Make the parent child connections
on both the sides

Deletion

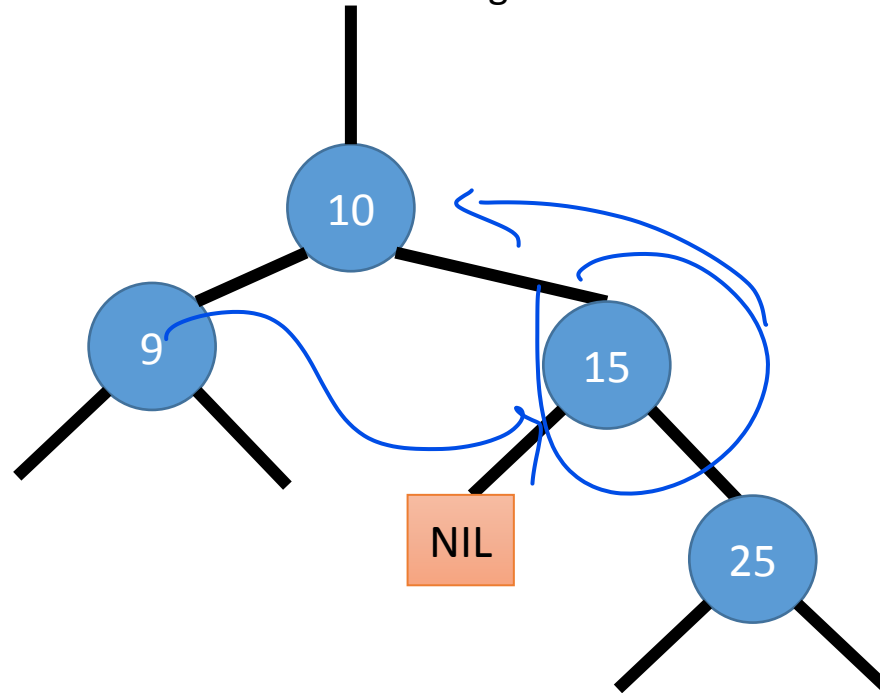
- Let us observe the following four cases:
- Case 3:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value is not the immediate child



Let the minimum node be y
Transplant y with its right child.
Transplant z by y
Make the parent child connections
on both the sides

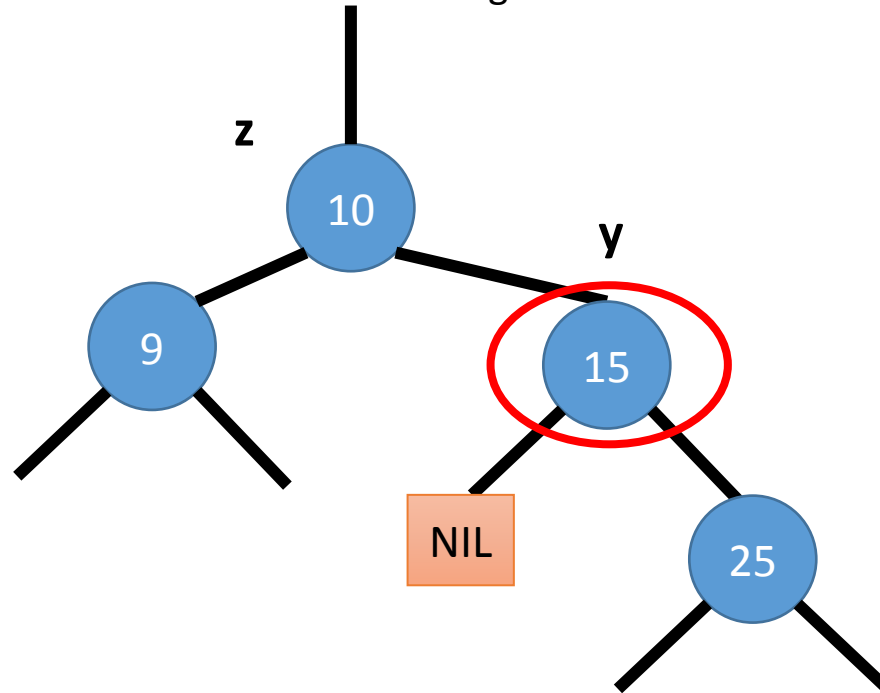
Deletion

- Let us observe the following four cases:
- Case 4:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value **is the immediate child**



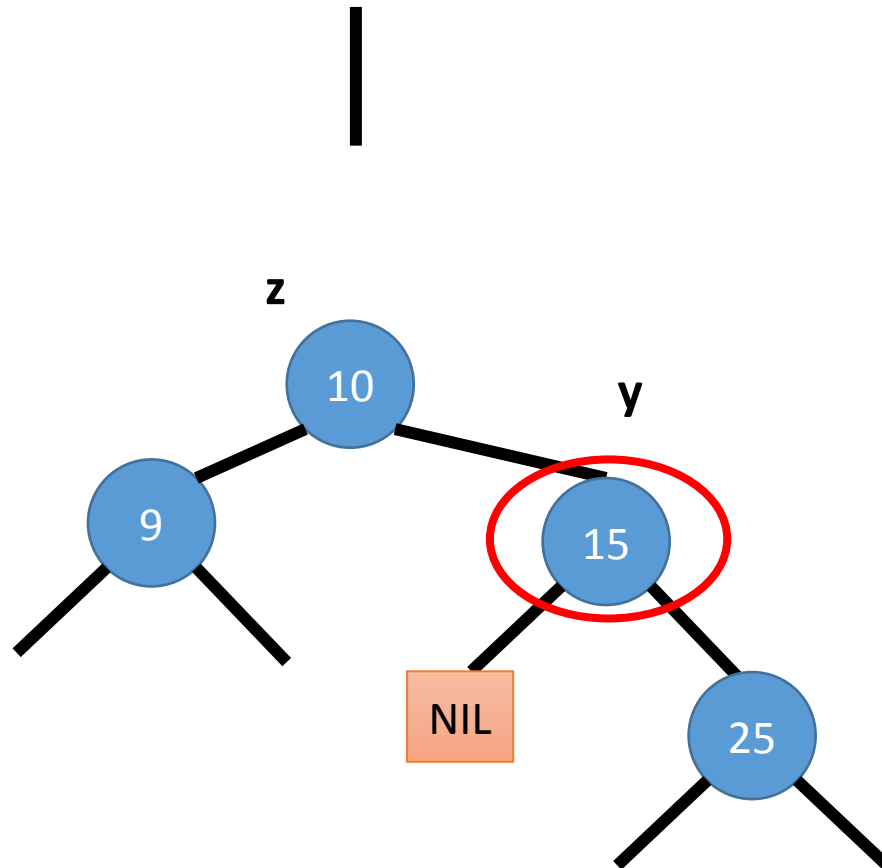
Deletion

- Let us observe the following four cases:
- Case 4:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value **is the immediate child**



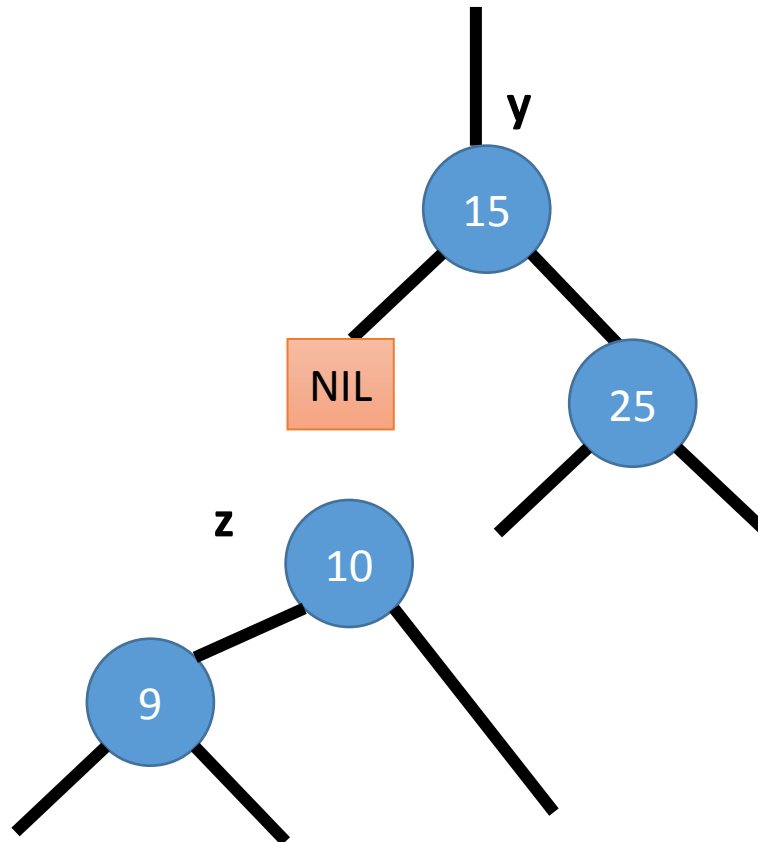
Deletion

- Let us observe the following four cases:
- Case 4:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value **is the immediate child**



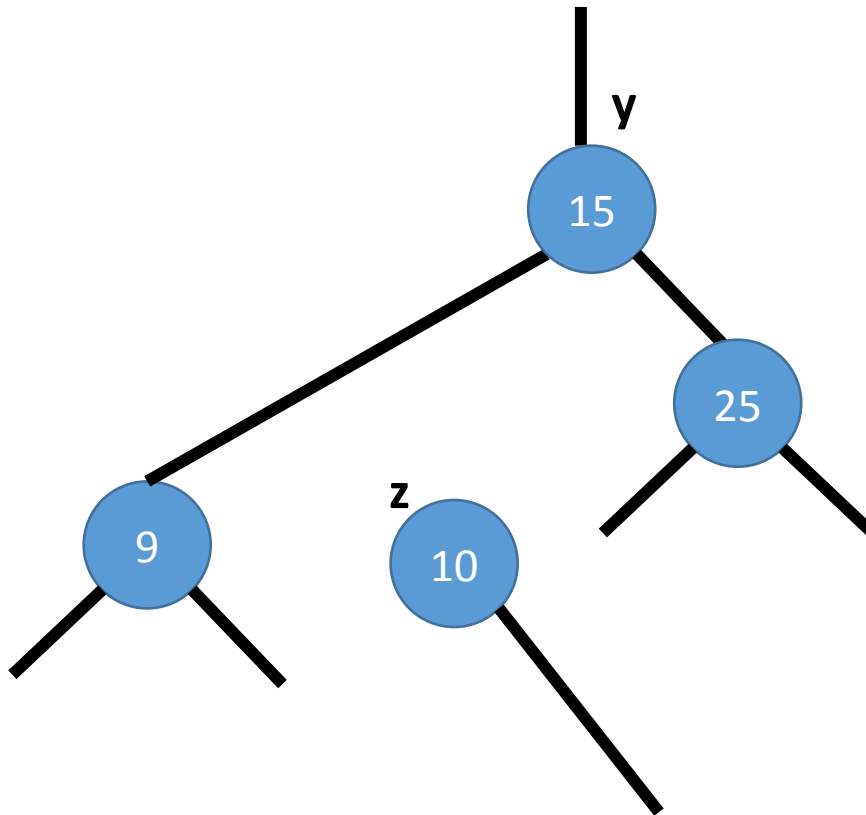
Deletion

- Let us observe the following four cases:
- Case 4:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value **is the immediate child**



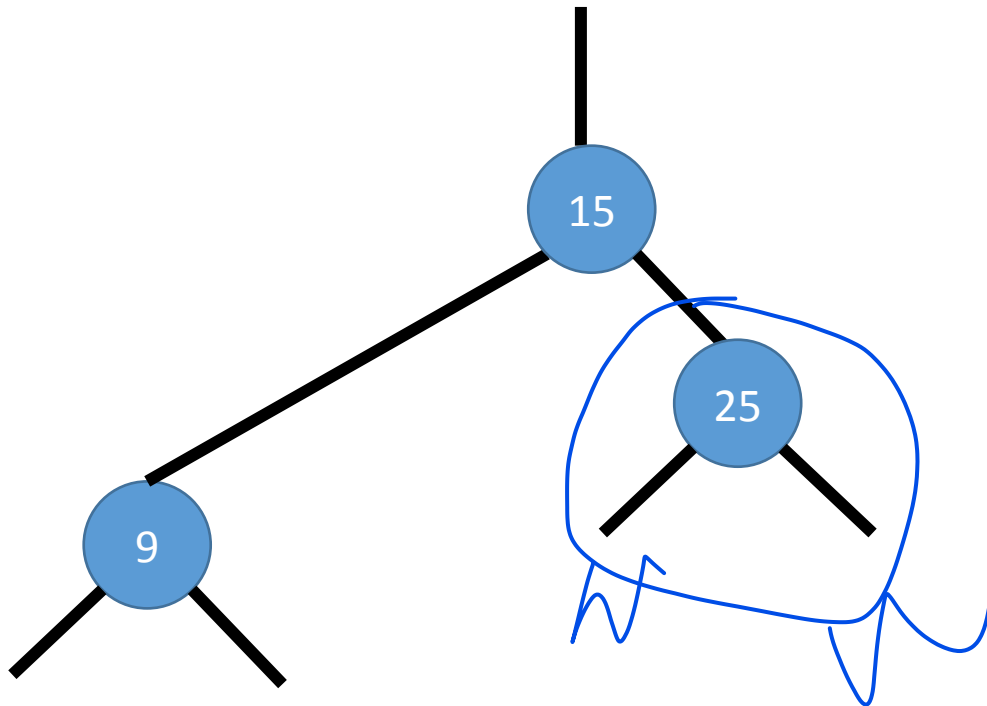
Deletion

- Let us observe the following four cases:
- Case 4:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value **is the immediate child**

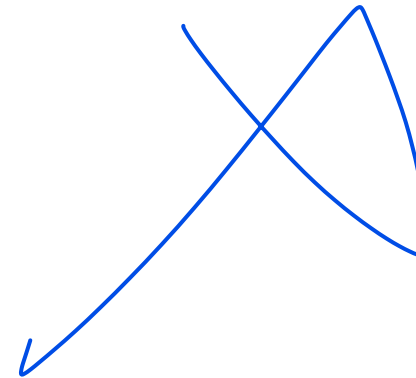


Deletion

- Let us observe the following four cases:
- Case 4:
 - Node z (**node to be deleted**) has both children
 - We need to find the minimum child on the right sub tree and the minimum value **is the immediate child**



Let us write the pseudocode for delete



Let us write the pseudocode for delete

TREE-DELETE(T, z)

```
1  if  $z.left == \text{NIL}$ 
2      TRANSPLANT( $T, z, z.right$ )
3  elseif  $z.right == \text{NIL}$ 
4      TRANSPLANT( $T, z, z.left$ )
5  else  $y = \text{TREE-MINIMUM}(z.right)$ 
6      if  $y.p \neq z$ 
7          { TRANSPLANT( $T, y, y.right$ )
8               $y.right = z.right$ 
9               $y.right.p = y$ 
10         TRANSPLANT( $T, z, y$ )
11          $y.left = z.left$ 
12          $y.left.p = y$ 
```

1 child

Successor

Time Complexity

- The time required for each of the operations is $O(h)$.
- Because all the operations require pointers to climb down.
 - SEARCH – climbs down the height h
 - MINIMUM – climbs down the height h
 - MAXIMUM – climbs down the height h
 - PREDECESSOR – calls MAXIMUM
 - SUCCESSOR – calls MINIMUM
 - INSERT – climbs down the height h
 - DELETE – requires a call to MINIMUM – h
- What is the value of h ?

$h=O(\log n)$

- The above equation is applicable only if the tree is balanced.
- Let us prove it.

Thank You

